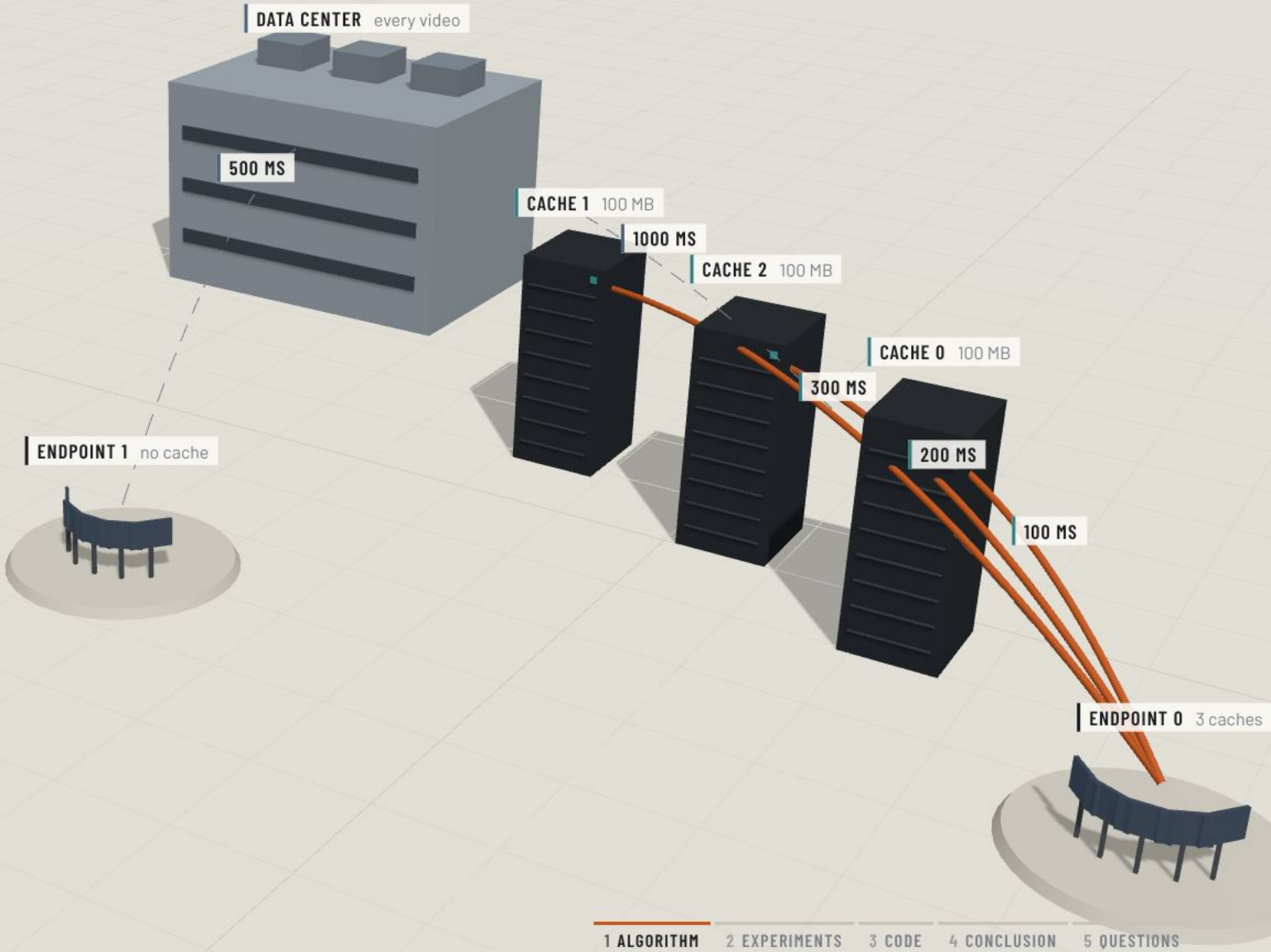
GOOGLE HASH CODE 2017 · QUALIFICATION ROUND

STREAMING VIDEOS

From a fast greedy to proven optimal answers

GROUP D Kacem · Daniel · Marouane · Dhia

Advanced Problem Solving



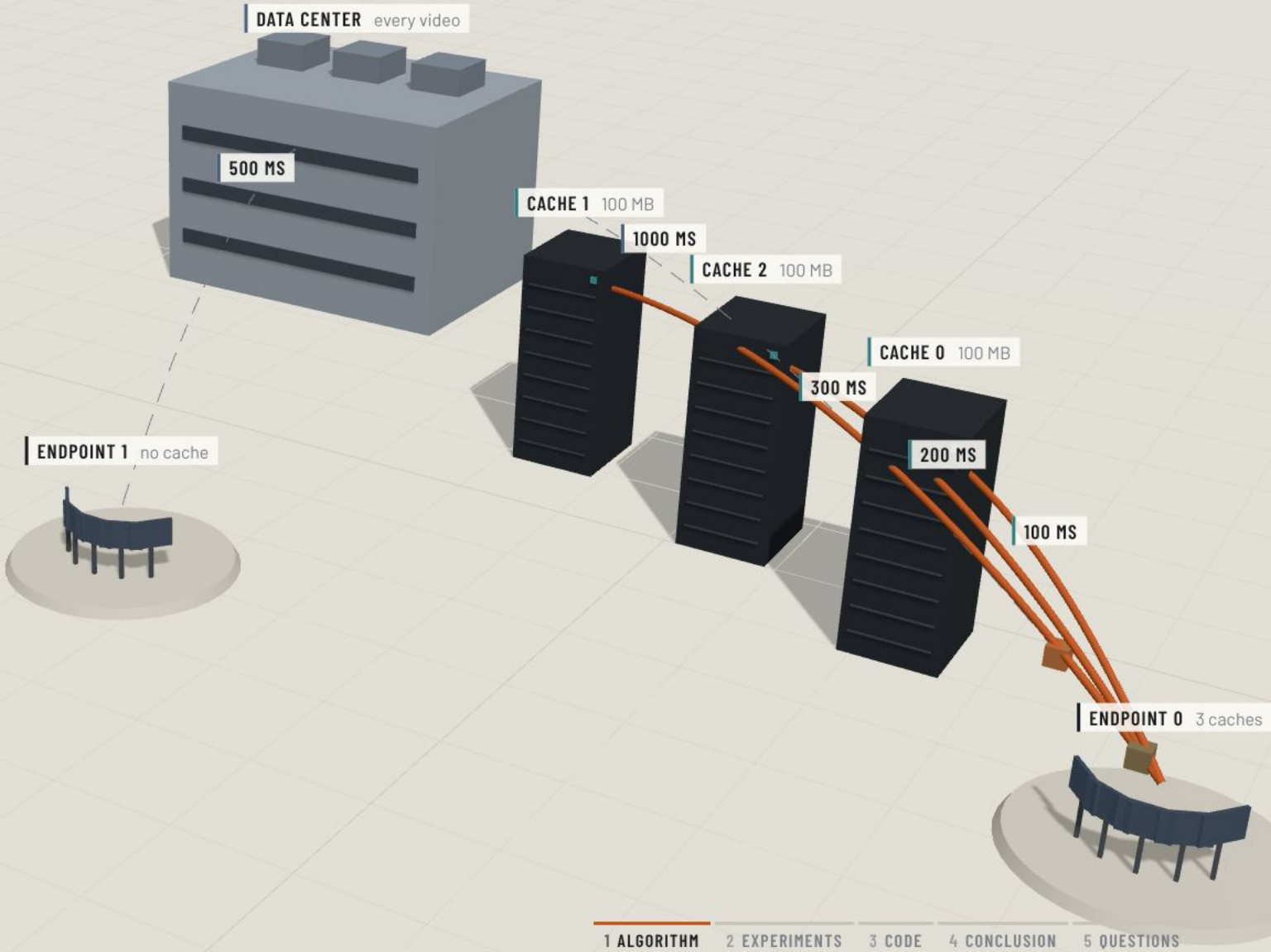
Statement · example network (3D view is ours)

EVERY VIDEO LIVES FAR AWAY

Without caches, every request goes to the **data center** and back. That is slow.

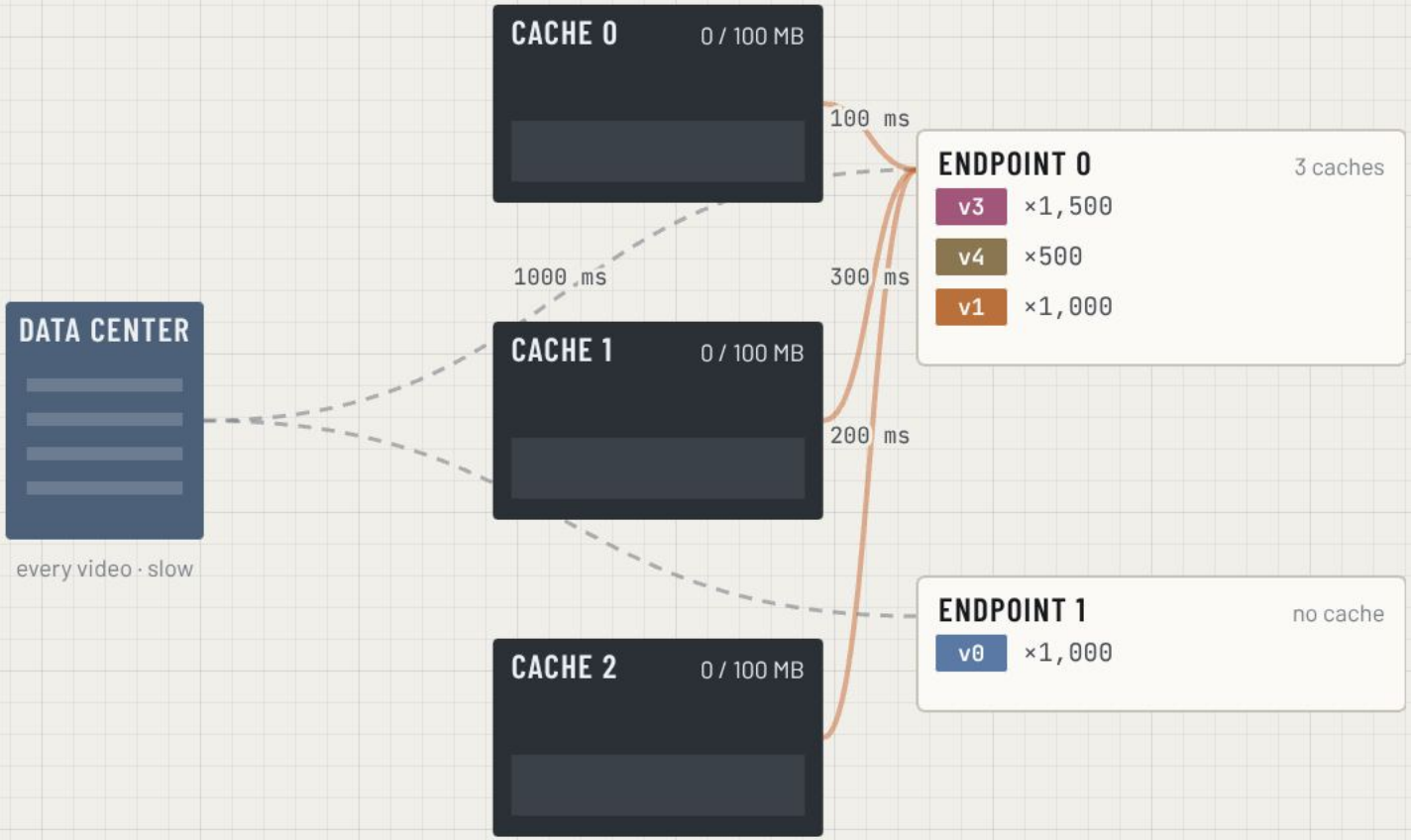
- cache link · fast (100–300 ms)
- data-center route · slow (500–1000 ms)

Coloured cubes = requests, one colour per video. Cable length follows latency.



Statement · example network (3D view is ours)

02 THE PROBLEM IN ONE MINUTE

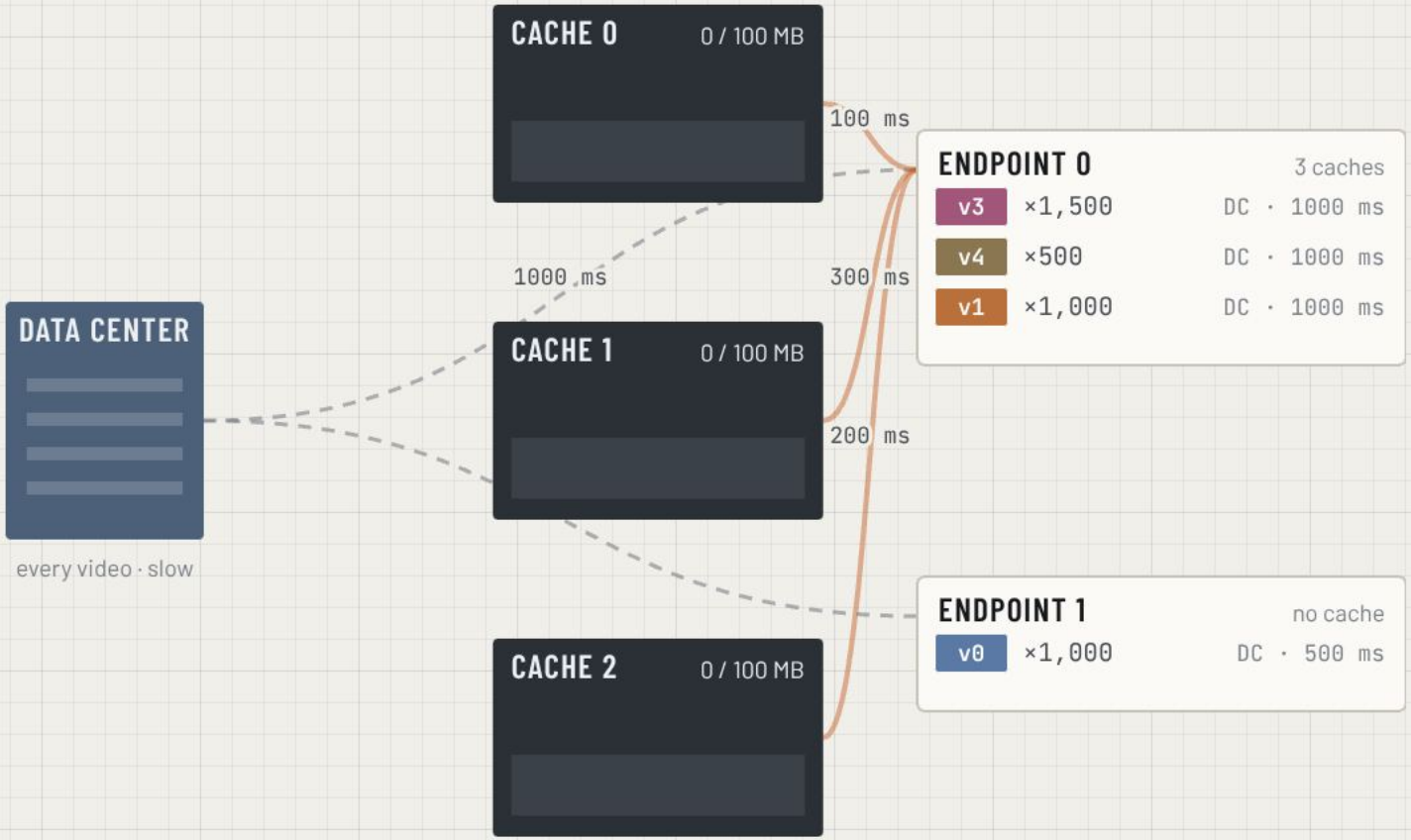


FROM THE STATEMENT

5 videos, **3 caches of 100 MB**, 2 user groups (endpoints). Endpoint 1 has no cache at all.

Statement · example input, example submission, scoring

02 THE PROBLEM IN ONE MINUTE



FROM THE STATEMENT

5 videos, **3 caches of 100 MB**, 2 user groups (endpoints). Endpoint 1 has no cache at all.

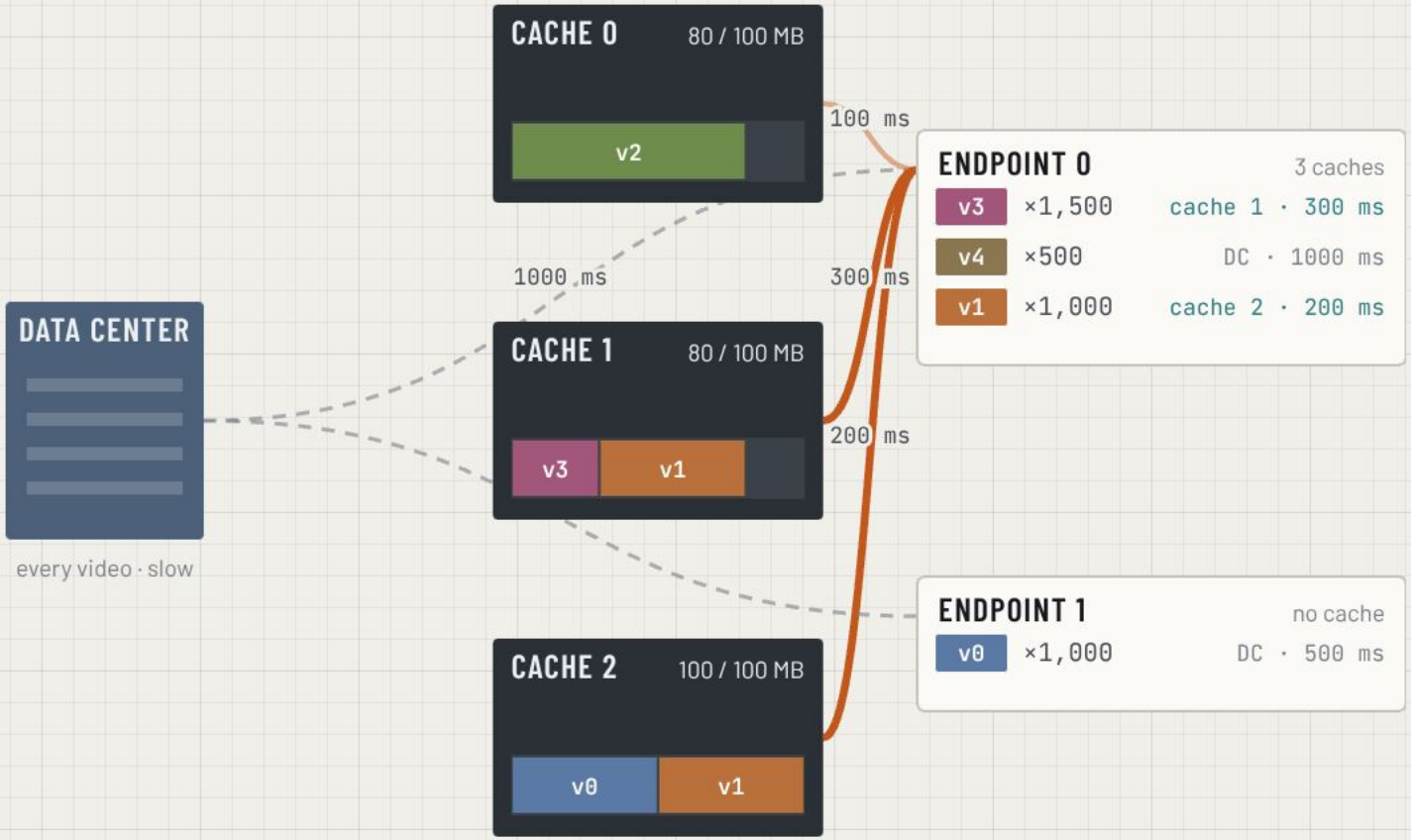
OUR CHOICE

Which videos to copy into which caches. A video may sit in several caches, or none.

Each request is served by the **fastest** place that holds the video.

Statement · example input, example submission, scoring

02 THE PROBLEM IN ONE MINUTE



FROM THE STATEMENT

5 videos, **3 caches of 100 MB**, 2 user groups (endpoints). Endpoint 1 has no cache at all.

OUR CHOICE

Which videos to copy into which caches. A video may sit in several caches, or none.

Each request is served by the **fastest** place that holds the video.

THE SCORE

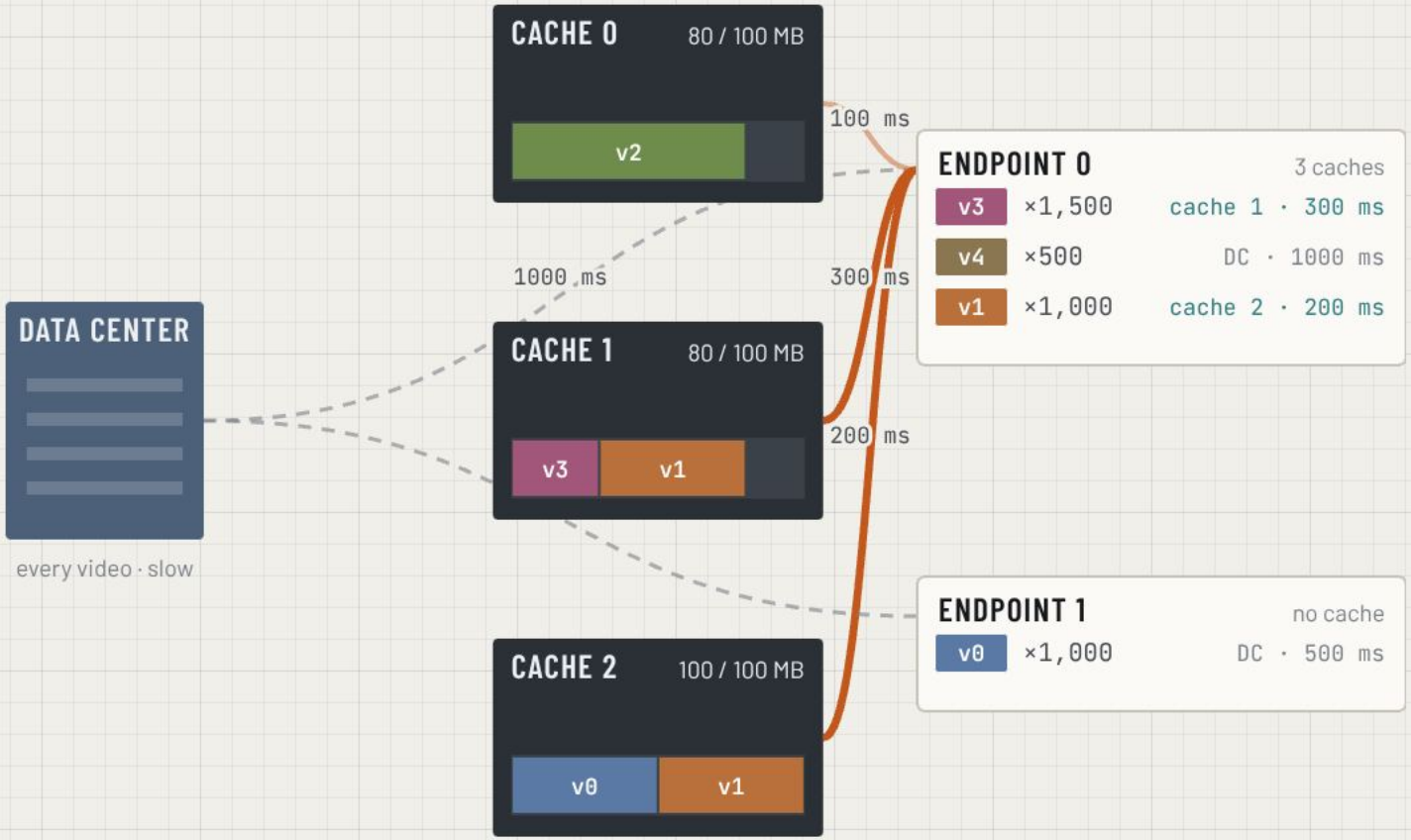
$$\text{score} = 1000 \times \frac{\Sigma \text{ requests} \times \text{time saved}}{\Sigma \text{ requests}}$$

The score is the average time saved per request, in microseconds.

Statement's answer **462,500**

Statement · example input, example submission, scoring

02 THE PROBLEM IN ONE MINUTE



FROM THE STATEMENT

5 videos, **3 caches of 100 MB**, 2 user groups (endpoints). Endpoint 1 has no cache at all.

OUR CHOICE

Which videos to copy into which caches. A video may sit in several caches, or none.

Each request is served by the **fastest** place that holds the video.

THE SCORE

$$\text{score} = 1000 \times \frac{\Sigma \text{ requests} \times \text{time saved}}{\Sigma \text{ requests}}$$

The score is the average time saved per request, in microseconds.

Statement's answer **462,500**

CAN WE BEAT 462,500?

Statement · example input, example submission, scoring

03 WHY IT IS HARD, AND OUR PLAN

EVERY CACHE IS A KNAPSACK

Videos have different sizes and a cache has a fixed number of MB. Choosing the best set for **one** cache is already NP-hard.

THE CACHES INTERACT

A copy is only worth the time it saves **beyond** what the users already get. Video A in cache 1 can make its copy in cache 2 worthless.

Statement · input limits · our pipeline (PIPELINE.md)

03 WHY IT IS HARD, AND OUR PLAN

EVERY CACHE IS A KNAPSACK

Videos have different sizes and a cache has a fixed number of MB. Choosing the best set for **one** cache is already NP-hard.

THE CACHES INTERACT

A copy is only worth the time it saves **beyond** what the users already get. Video A in cache 1 can make its copy in cache 2 worthless.

10,000
videos (at most)

1,000
caches

1,000,000
request lines

No method is both **exact** and **fast** at this size. So we mix both.

Statement · input limits · our pipeline (PIPELINE.md)

03 WHY IT IS HARD, AND OUR PLAN

OUR PIPELINE, BOTTOM TO TOP

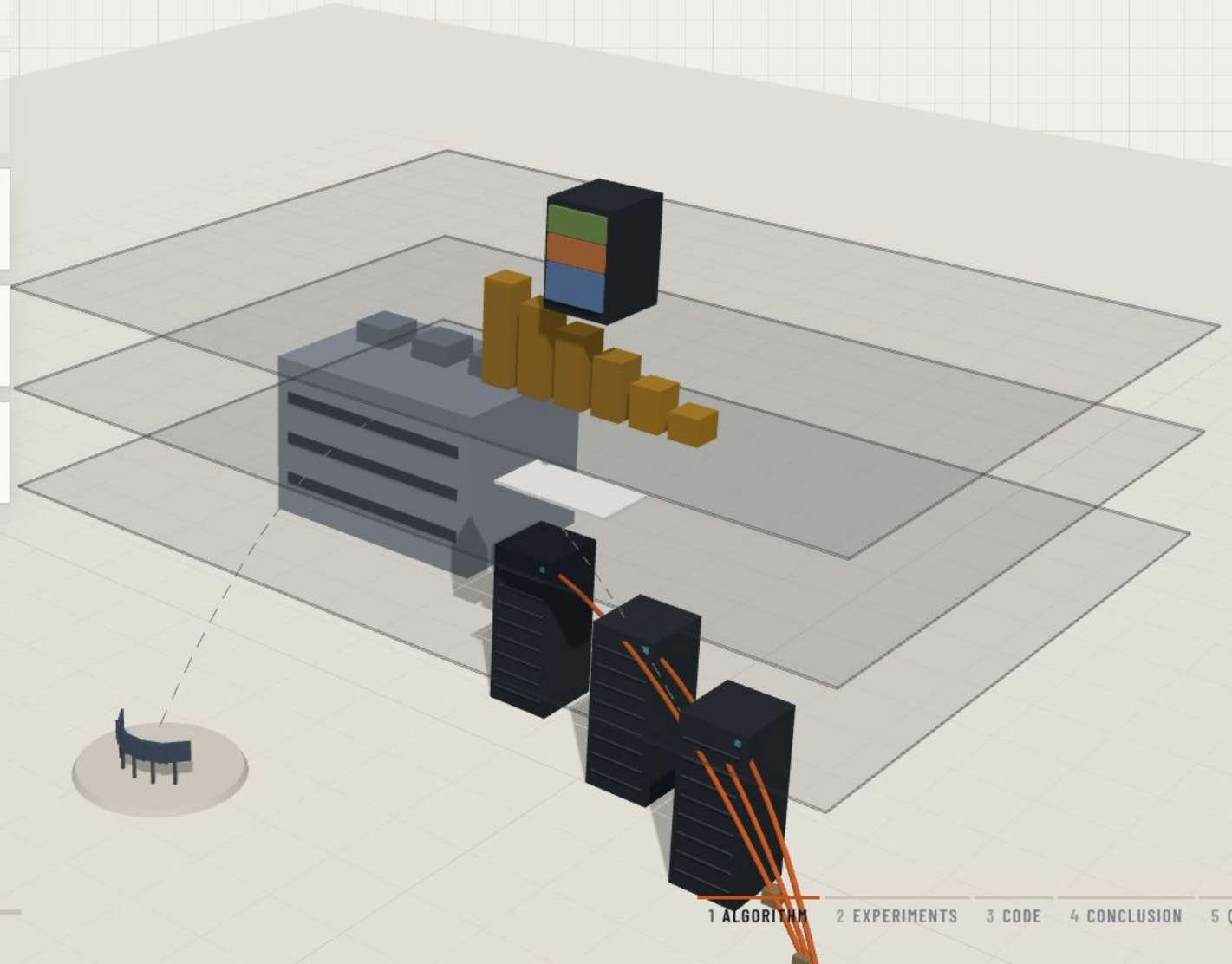
05 · EXACT SOLVER
small instances: proven optimum · large: a bound

04 · LARGE NEIGHBOURHOOD SEARCH
empty two linked caches, rebuild them together

03 · EXACT RE-PACK
freeze the others, fill one cache optimally

02 · GREEDY FILL
best saving per MB first, then evict and swap

01 · READ & INDEX
each cache only looks at videos its own users ask for



Statement · input limits · our pipeline (PIPELINE.md)

03 WHY IT IS HARD, AND OUR PLAN

OUR PIPELINE, BOTTOM TO TOP

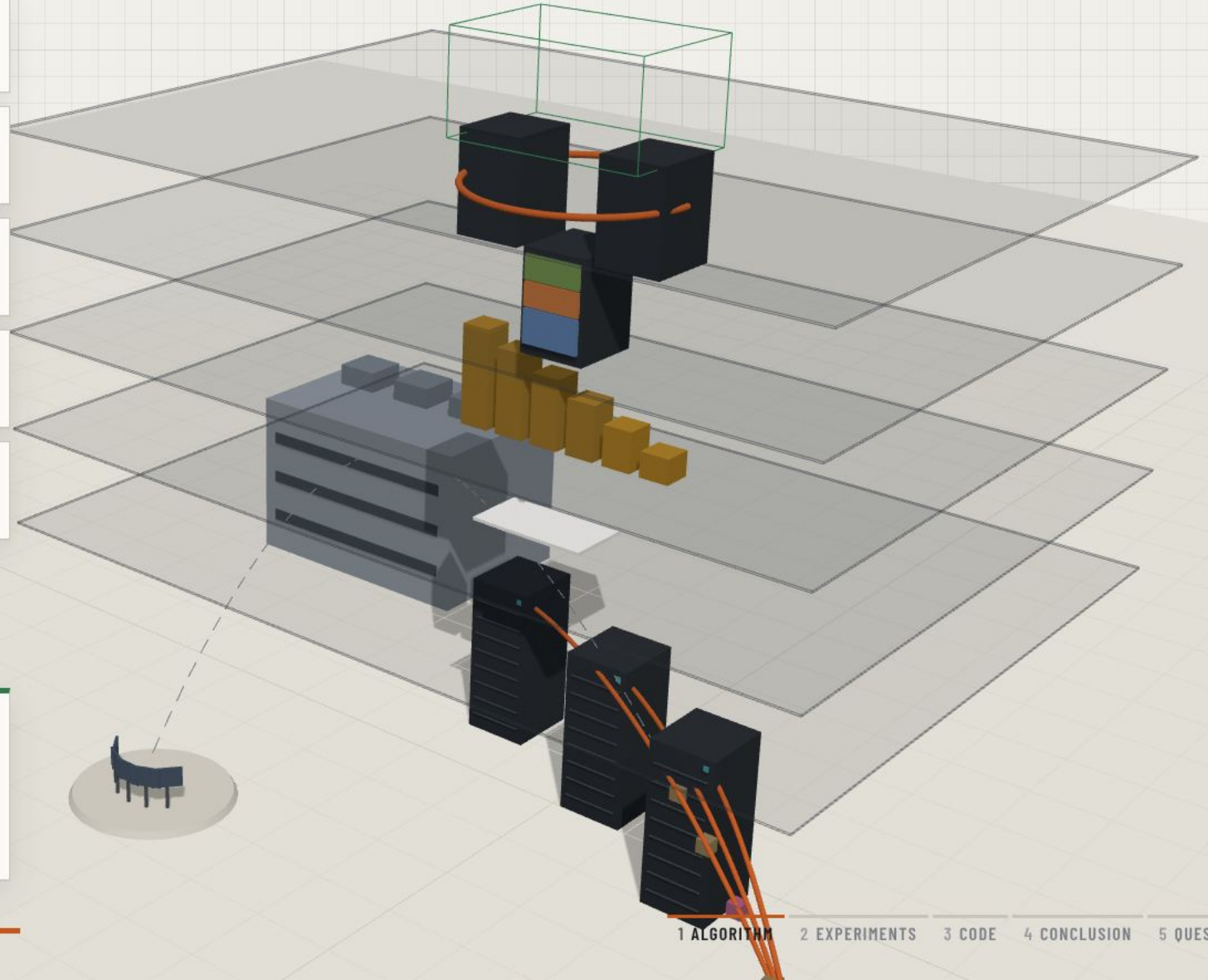
05 · EXACT SOLVER
small instances: proven optimum · large: a bound

04 · LARGE NEIGHBOURHOOD SEARCH
empty two linked caches, rebuild them together

03 · EXACT RE-PACK
freeze the others, fill one cache optimally

02 · GREEDY FILL
best saving per MB first, then evict and swap

01 · READ & INDEX
each cache only looks at videos its own users ask for

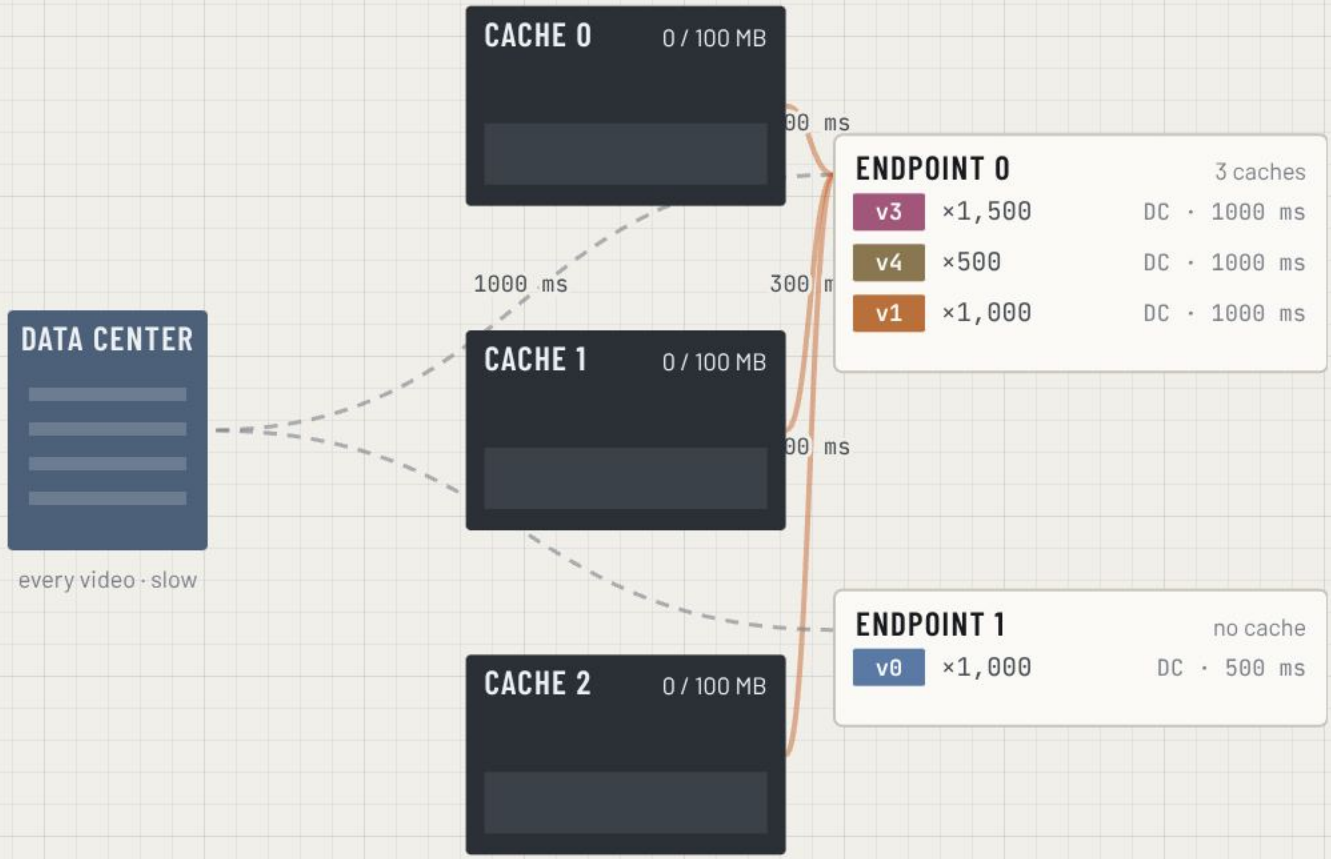


NO STEP CAN MAKE IT WORSE

Each floor starts from the answer below it and keeps a change only if the score does not drop.

Statement · input limits · our pipeline (PIPELINE.md)

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	
2	2	v3	1,200,000	40,000	
3	1	v3	1,050,000	35,000	
4	0	v1	900,000	18,000	
5	2	v1	800,000	16,000	
6	1	v1	700,000	14,000	

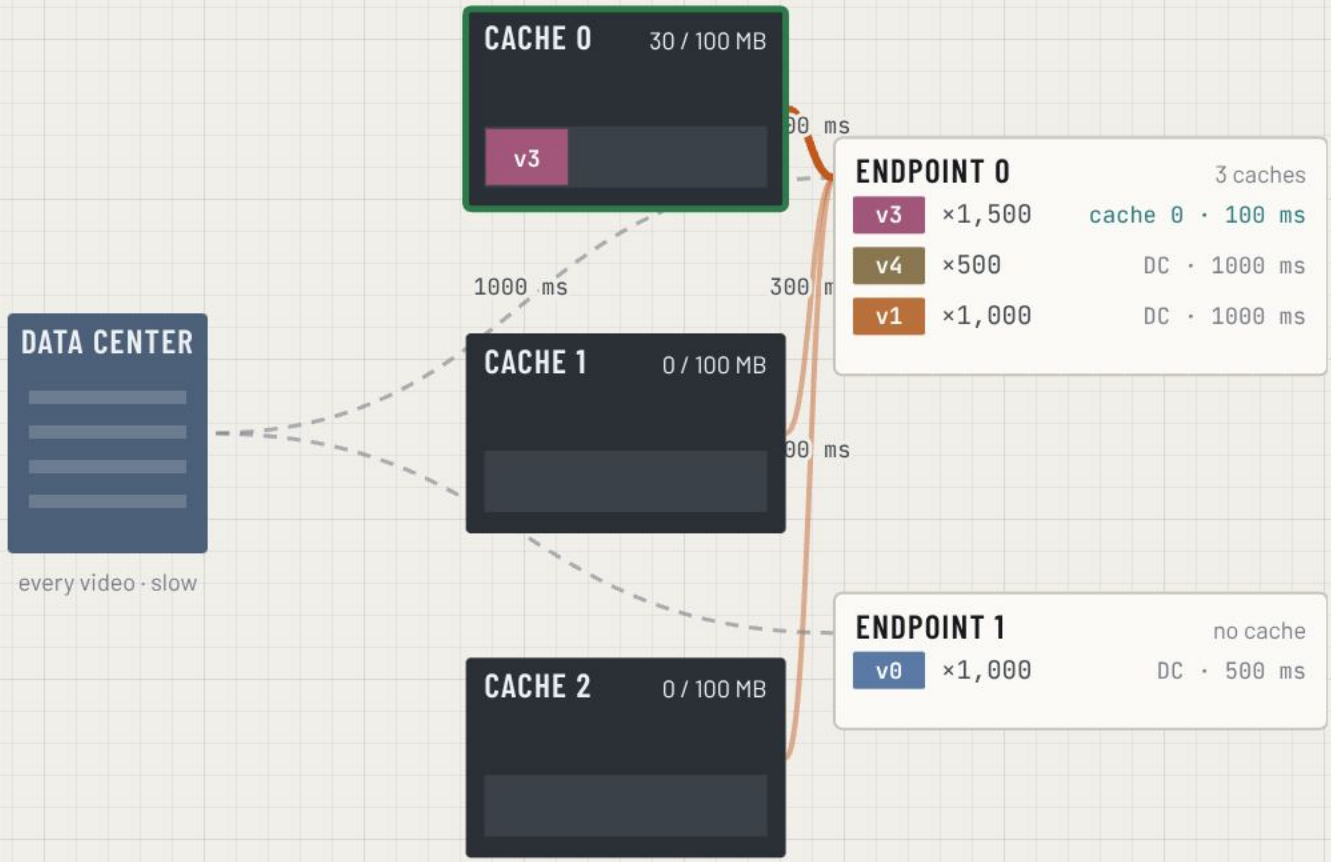
Score every pair once. Cache 0 · video 3: 1,500 requests × (1,000 – 100) ms = 1,350,000 ms, ÷ 30 MB = **45,000 per MB**.

Video 4 (110 MB) never fits a 100 MB cache. Endpoint 1 has no cache, so its request never enters the queue.

Score **0**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

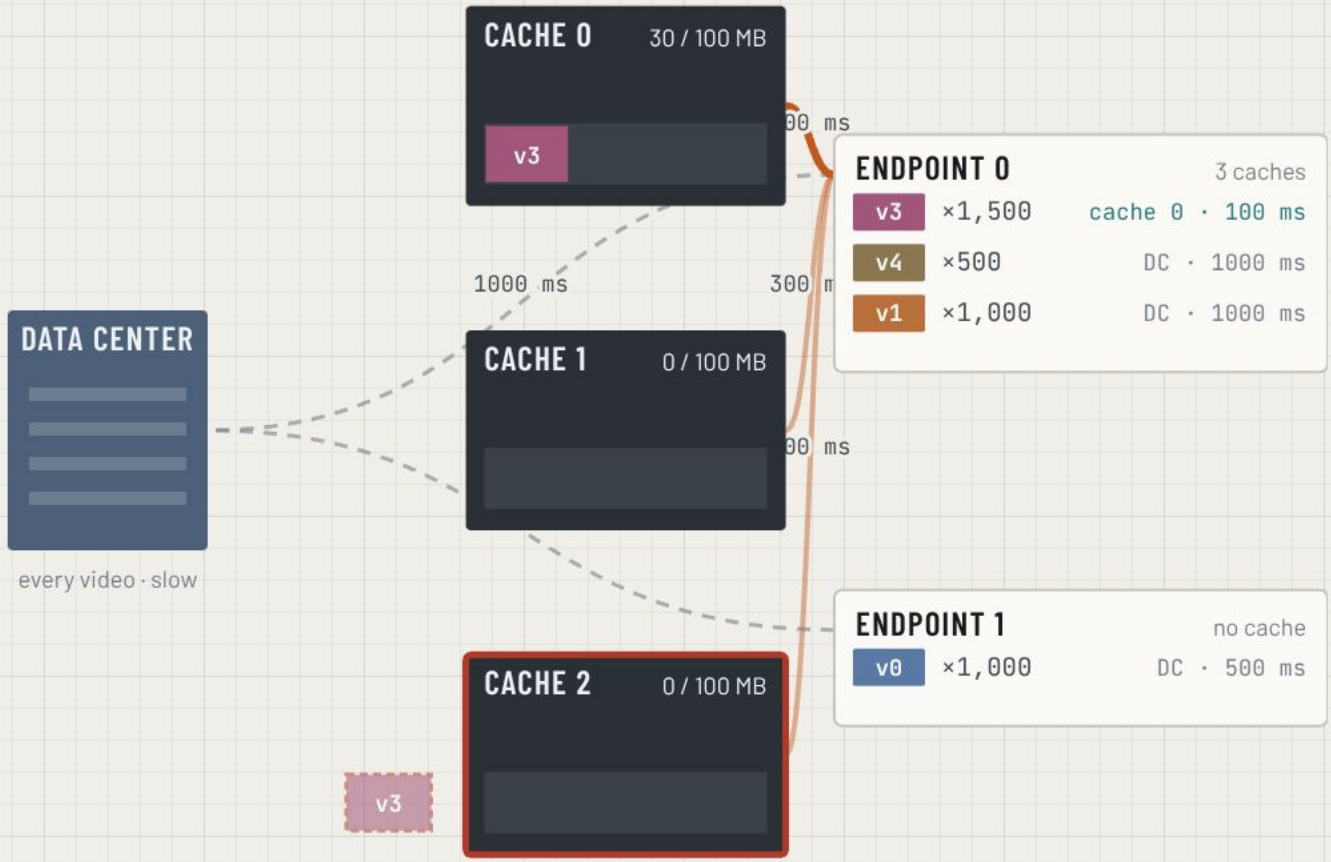
#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
2	2	v3	1,200,000	40,000	
3	1	v3	1,050,000	35,000	
4	0	v1	900,000	18,000	
5	2	v1	800,000	16,000	
6	1	v1	700,000	14,000	

Pop 1: cache 0 · video 3. Re-check the saving: still 1,350,000.
Unchanged → place it.

Score **337,500**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

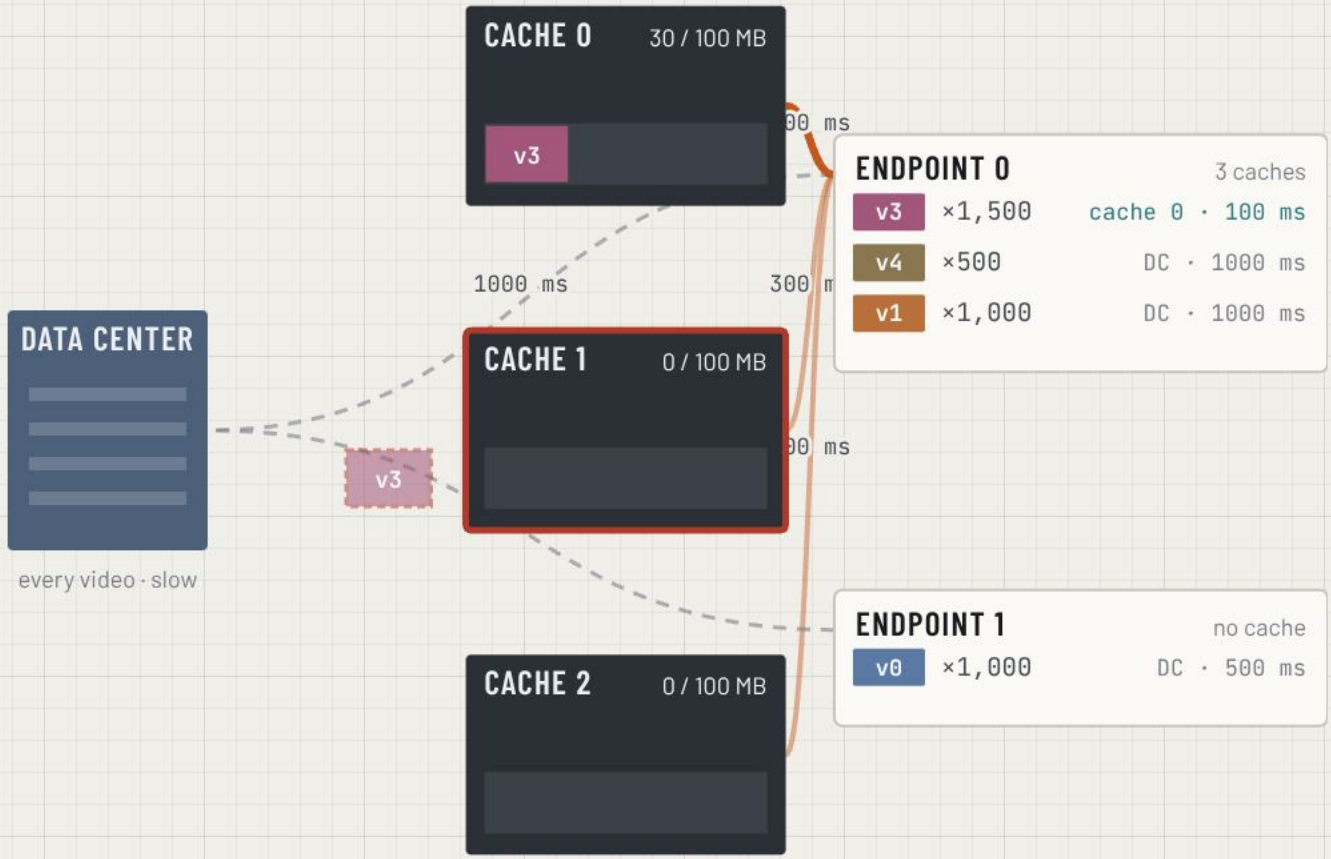
#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	~~DROP~~
3	1	v3	1,050,000	35,000	
4	0	v1	900,000	18,000	
5	2	v1	800,000	16,000	
6	1	v1	700,000	14,000	

Pop 2: cache 2 · video 3. Re-check: endpoint 0 already gets video 3 from cache 0 in 100 ms. Saving now 0. **Drop it.**

Score **337,500**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

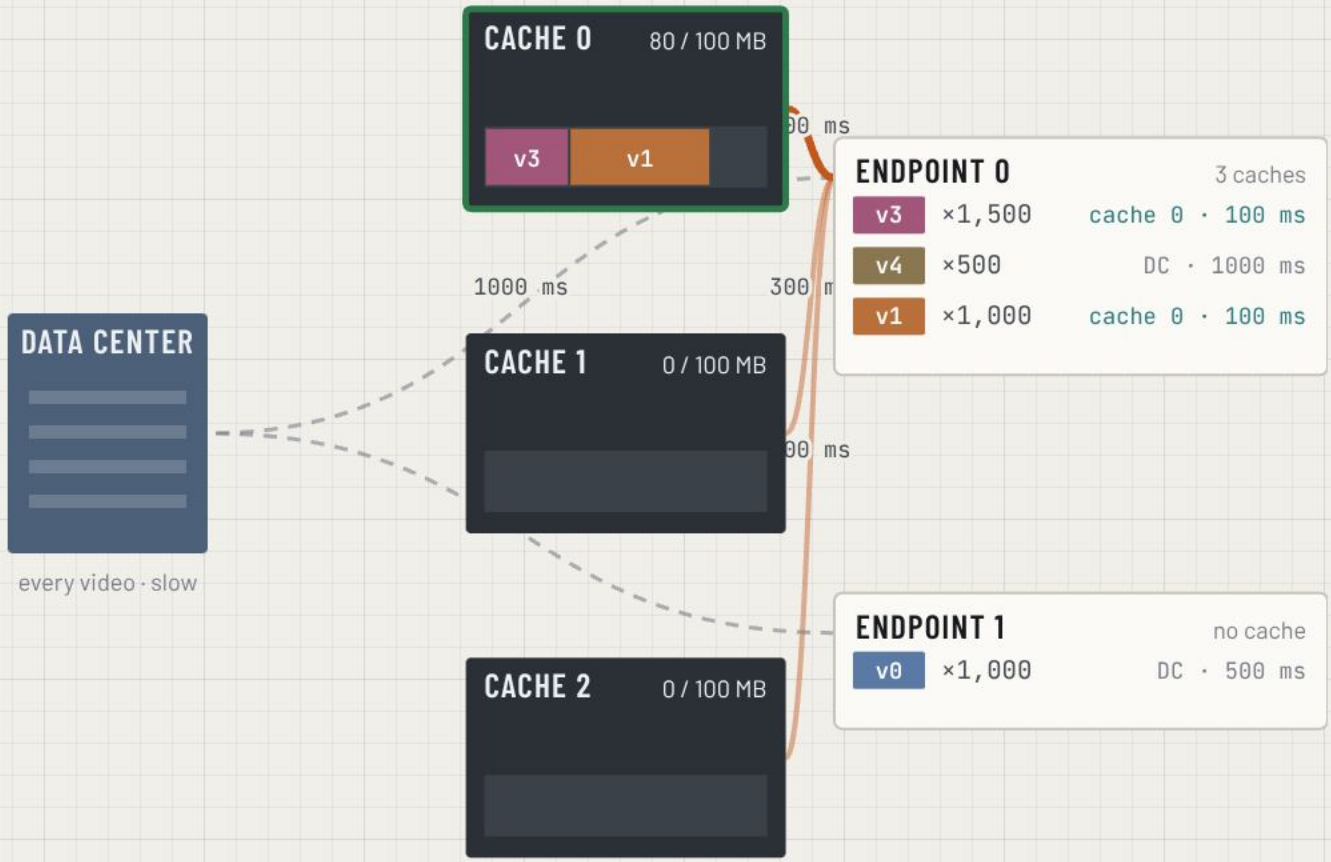
#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	~~DROP~~
~~3~~	~~1~~	v3	~~1,050,000~~	~~35,000~~	~~DROP~~
4	0	v1	900,000	18,000	
5	2	v1	800,000	16,000	
6	1	v1	700,000	14,000	

Pop 3: cache 1 · video 3. Re-check: endpoint 0 already gets video 3 from cache 0 in 100 ms. Saving now 0. **Drop it.**

Score **337,500**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

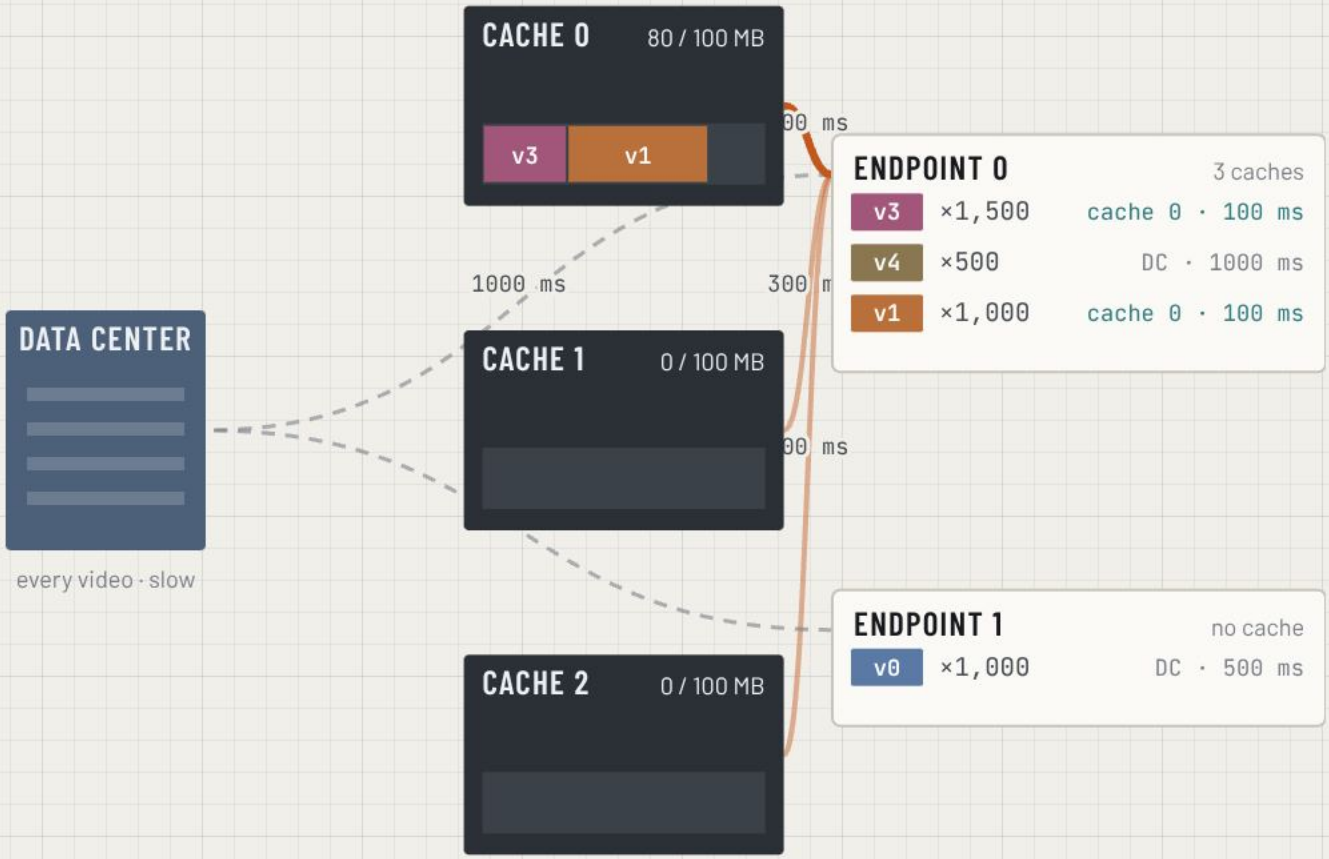
#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	~~DROP~~
~~3~~	~~1~~	v3	~~1,050,000~~	~~35,000~~	~~DROP~~
4	0	v1	900,000	18,000	PLACE
5	2	v1	800,000	16,000	
6	1	v1	700,000	14,000	

Pop 4: cache 0 · video 1. Re-check the saving: still 900,000.
Unchanged → place it.

Score **562,500**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

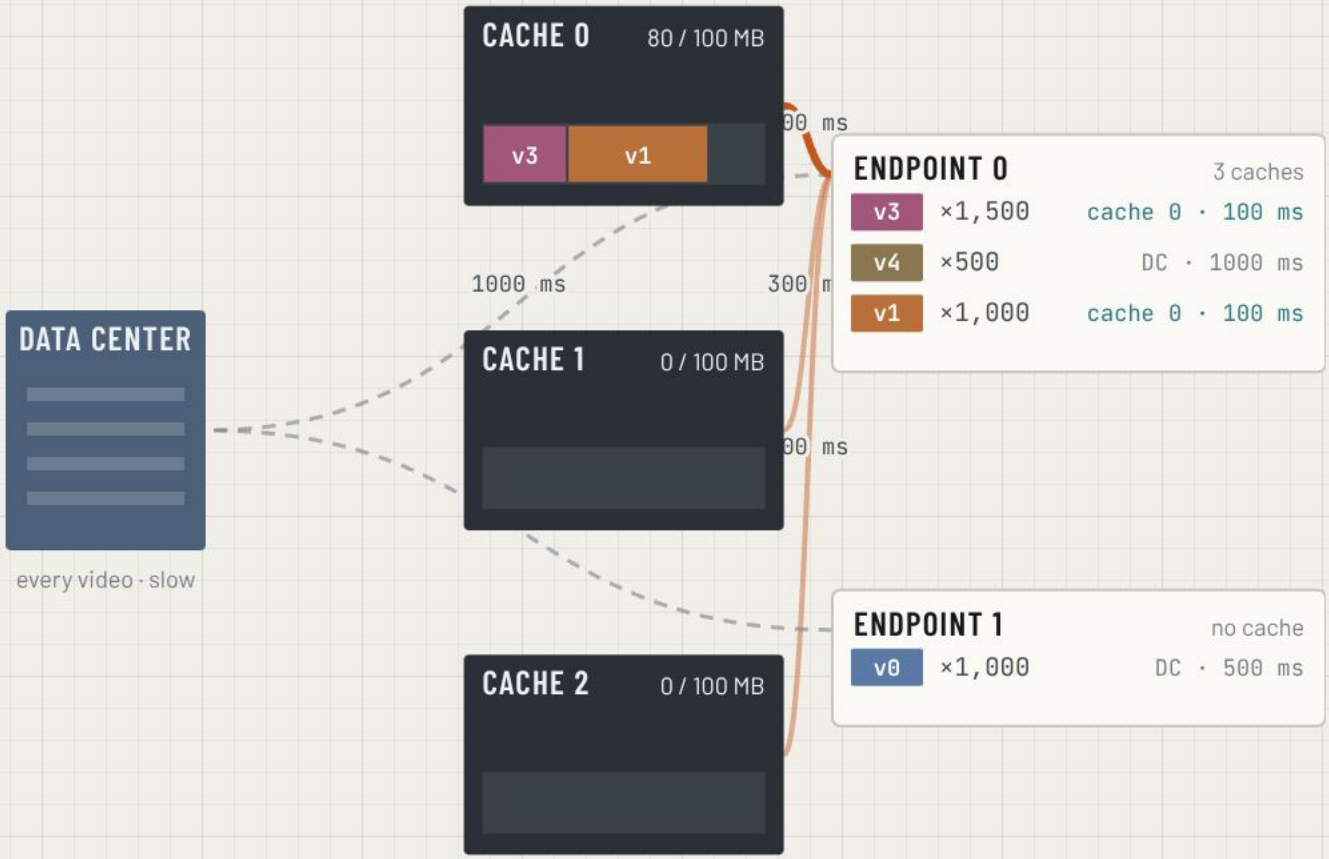
#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	DROP
~~3~~	~~1~~	v3	~~1,050,000~~	~~35,000~~	DROP
4	0	v1	900,000	18,000	PLACE
~~5~~	~~2~~	v1	~~800,000~~	~~16,000~~	DROP
~~6~~	~~1~~	v1	~~700,000~~	~~14,000~~	DROP

Pops 5–6: video 1 in caches 2 and 1. Endpoint 0 already gets it from cache 0 in 100 ms. **Both dropped.**

Score **562,500**

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	~~DROP~~
~~3~~	~~1~~	v3	~~1,050,000~~	~~35,000~~	~~DROP~~
4	0	v1	900,000	18,000	PLACE
~~5~~	~~2~~	v1	~~800,000~~	~~16,000~~	~~DROP~~
~~6~~	~~1~~	v1	~~700,000~~	~~14,000~~	~~DROP~~

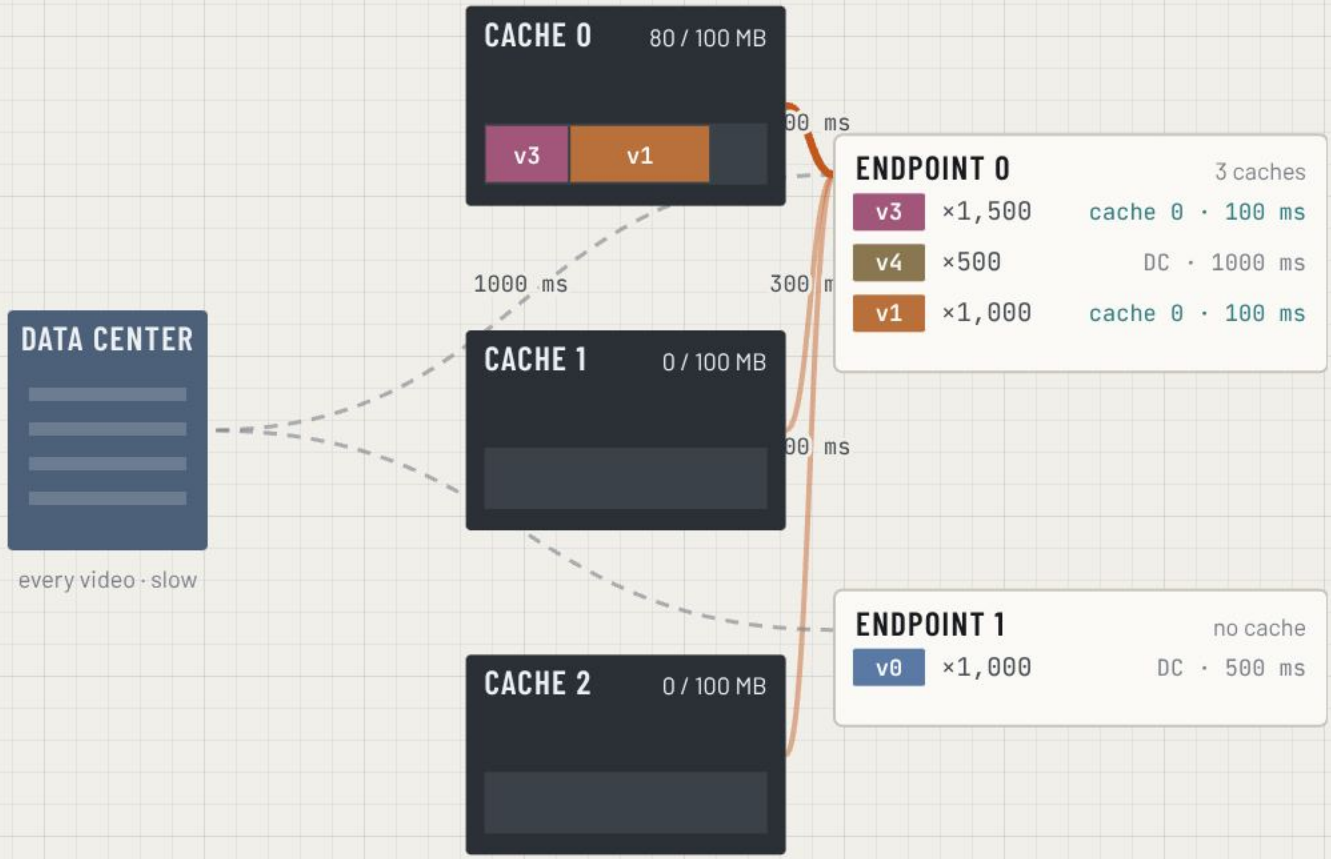
Queue empty, round over. Evict: no copy saves nothing. Swap: no better video fits. The next round places nothing, so we stop.

Score **562,500**

+100,000 VS THE STATEMENT

Statement's example · run live by our solver

04 RUNNING IT ON THE EXAMPLE, POP BY POP



THE QUEUE · BEST SAVING PER MB FIRST

#	CACHE	VIDEO	SAVING (MS)	PER MB	DECISION
1	0	v3	1,350,000	45,000	PLACE
~~2~~	~~2~~	v3	~~1,200,000~~	~~40,000~~	~~DROP~~
~~3~~	~~1~~	v3	~~1,050,000~~	~~35,000~~	~~DROP~~
4	0	v1	900,000	18,000	PLACE
~~5~~	~~2~~	v1	~~800,000~~	~~16,000~~	~~DROP~~
~~6~~	~~1~~	v1	~~700,000~~	~~14,000~~	~~DROP~~

Is this the best possible? Endpoint 0 gets videos 3 and 1 from its fastest cache; video 4 can never be cached; endpoint 1 has no cache. **Yes: our exact solver proves 562,500 is optimal.**

Score **562,500**

+100,000 VS THE STATEMENT

Statement's example · run live by our solver

05 WHEN "BEST PER MB" IS FOOLED

EVICT after every round

A copy that no longer saves anything (users get the video as fast from another cache) is removed. The score is unchanged and the space goes back into the next round.

SWAP after every round

The weakest copy is traded for an outside video when the trade saves strictly more. Up to 20 candidates, 50 swaps per cache.

Our pipeline, step 2 (PIPELINE.md §2) · numbers are an illustration

05 WHEN "BEST PER MB" IS FOOLED

ONE CACHE · 10 MB ILLUSTRATION

VIDEO	SIZE	SAVES	PER MB
A	6 MB	600	100
B	5 MB	450	90
C	5 MB	450	90

GREEDY, BEST PER MB FIRST



Takes A, then nothing else fits → **600**

Our pipeline, step 2 (PIPELINE.md §2) · numbers are an illustration

05 WHEN "BEST PER MB" IS FOOLED

ONE CACHE · 10 MB ILLUSTRATION

VIDEO	SIZE	SAVES	PER MB
A	6 MB	600	100
B	5 MB	450	90
C	5 MB	450	90

GREEDY, BEST PER MB FIRST



Takes A, then nothing else fits → **600**

Freeze every other cache. Then each video's value in this cache is a fixed number, so filling it is **exactly a 0/1 knapsack**, solved by dynamic programming.

	0	1	2	3	4	5	6	7	8	9	10
+ A (6 MB, 600)	0	0	0	0	0	0	600	600	600	600	600
+ B (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	600
+ C (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	900

Cell = best saving using the videos up to this row with this many MB. Each cell: skip the video, or take it and add its saving to the best answer with its size removed.

Our pipeline, step 2 (PIPELINE.md §2) · numbers are an illustration

05 WHEN "BEST PER MB" IS FOOLED

ONE CACHE · 10 MB ILLUSTRATION

VIDEO	SIZE	SAVES	PER MB
A	6 MB	600	100
B	5 MB	450	90
C	5 MB	450	90

GREEDY, BEST PER MB FIRST



Takes A, then nothing else fits → **600**

EXACT KNAPSACK



→ **B + C = 900 (+50%)**

Freeze every other cache. Then each video's value in this cache is a fixed number, so filling it is **exactly a 0/1 knapsack**, solved by dynamic programming.

	0	1	2	3	4	5	6	7	8	9	10
+ A (6 MB, 600)	0	0	0	0	0	0	600	600	600	600	600
+ B (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	600
+ C (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	900

Cell = best saving using the videos up to this row with this many MB. Each cell: skip the video, or take it and add its saving to the best answer with its size removed.

Our pipeline, step 2 (PIPELINE.md §2) · numbers are an illustration

05 WHEN "BEST PER MB" IS FOOLED

ONE CACHE · 10 MB ILLUSTRATION

VIDEO	SIZE	SAVES	PER MB
A	6 MB	600	100
B	5 MB	450	90
C	5 MB	450	90

GREEDY, BEST PER MB FIRST



Takes A, then nothing else fits → **600**

EXACT KNAPSACK



→ **B + C = 900 (+50%)**

TRY ANOTHER CACHE SIZE

Freeze every other cache. Then each video's value in this cache is a fixed number, so filling it is **exactly a 0/1 knapsack**, solved by dynamic programming.

	0	1	2	3	4	5	6	7	8	9	10
+ A (6 MB, 600)	0	0	0	0	0	0	600	600	600	600	600
+ B (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	600
+ C (5 MB, 450)	0	0	0	0	0	450	600	600	600	600	900

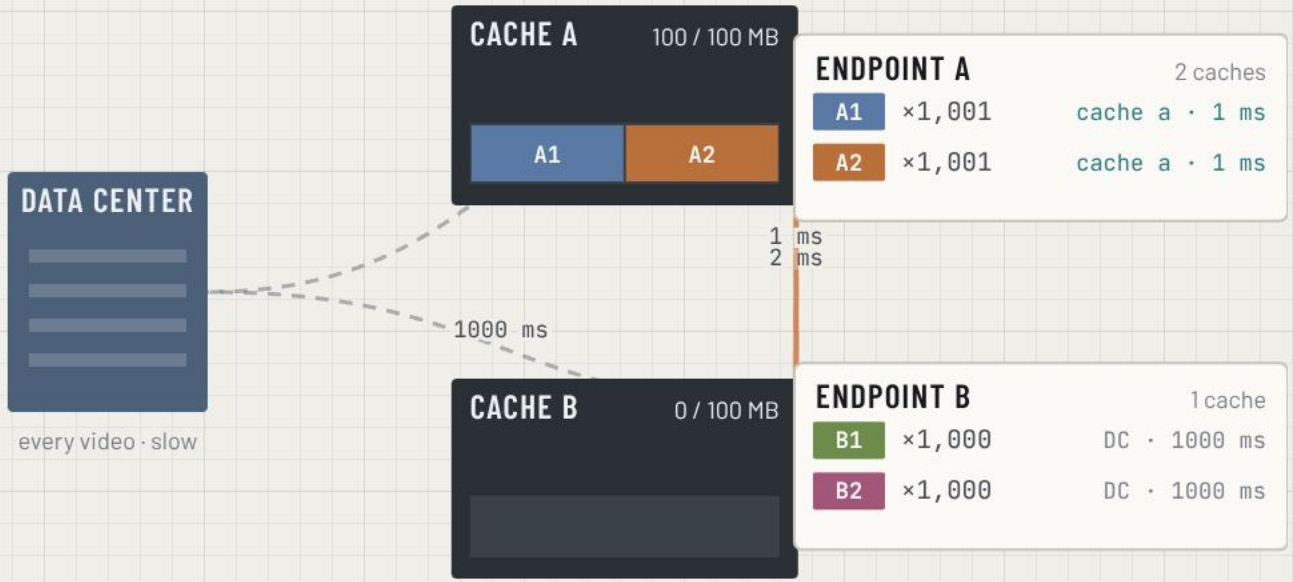
Cell = best saving using the videos up to this row with this many MB. Each cell: skip the video, or take it and add its saving to the best answer with its size removed.

REPEAT, CACHE AFTER CACHE

Until a full pass changes nothing. Every change raises the score, so it always stops. Its limit: it moves **one** cache at a time.

Our pipeline, step 2 (PIPELINE.md §2) · numbers are an illustration

06 TWO CACHES AT ONCE, THEN EXACT



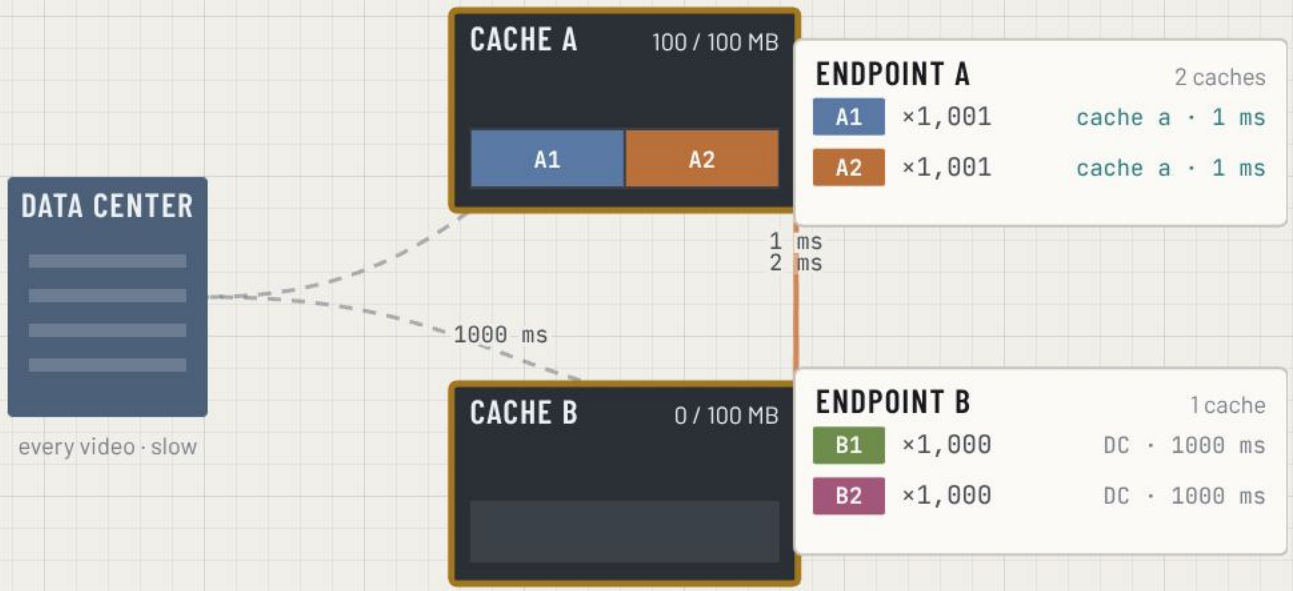
GREEDY

A's videos look a little better in cache a, so the greedy puts them there. Cache b can only reach A, who is already served. **B gets nothing.**

Score **499,749** 50.1% of the optimum

Trap gadget from our generator (models/trap_gadget.in) · solved live · real solver …

06 TWO CACHES AT ONCE, THEN EXACT



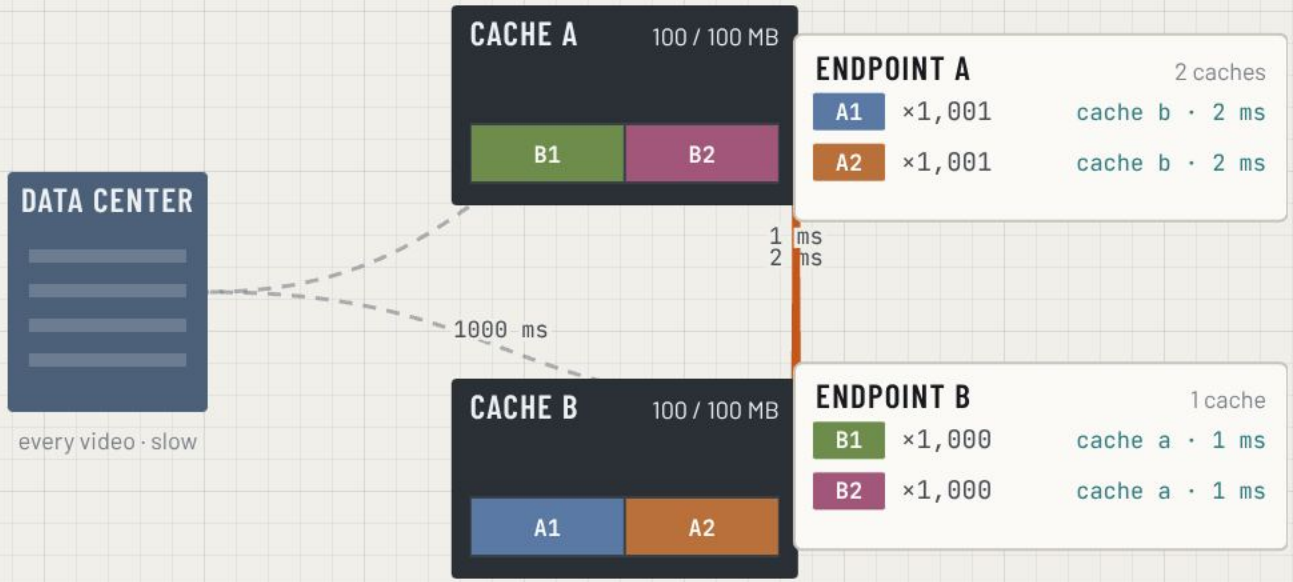
RE-PACK: STUCK

Re-pack **a** alone: A's videos are still worth most there. Re-pack **b** alone: A already gets them from a, so they are worth 0 in b. Each cache is optimal given the other.

Score **499,749** 50.1% of the optimum

Trap gadget from our generator (models/trap_gadget.in) · solved live · real solver …

06 TWO CACHES AT ONCE, THEN EXACT



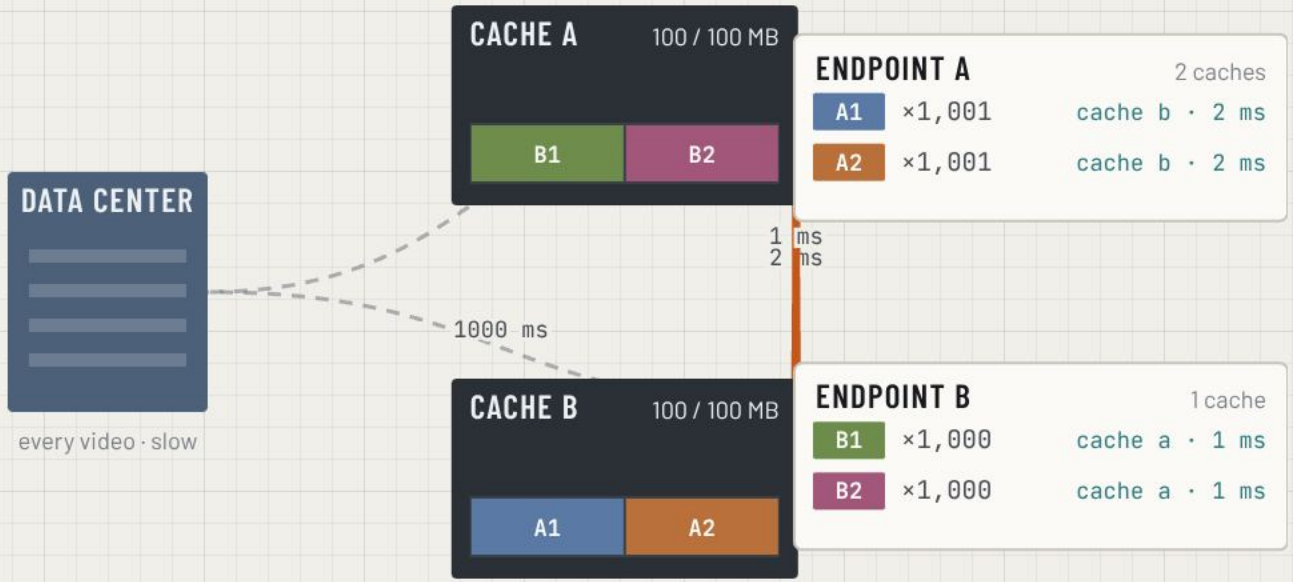
LARGE NEIGHBOURHOOD SEARCH

Pick a cache and a neighbour sharing users. **Empty both**, re-pack them together in random order, keep the result if the score did not drop. Repeat for the time budget.

Score **998,499** 100.0% of the optimum

Trap gadget from our generator (models/trap_gadget.in) · solved live · real solver …

06 TWO CACHES AT ONCE, THEN EXACT



Score **998,499** 100.0% of the optimum

THE EXACT SOLVER

$x_{c,v} \in \{0,1\}$: video v stored in cache c

$y_{r,c} \in [0,1]$: request r served by cache c

maximise $\Sigma\ n_r\ (L_D - L_c)\ y_{r,c}$

$\Sigma_c\ y_{r,c} \le 1\ \cdot\ y_{r,c} \le x_{c,v}\ \cdot\ \Sigma_v\ \text{size}_v\ x_{c,v} \le X$

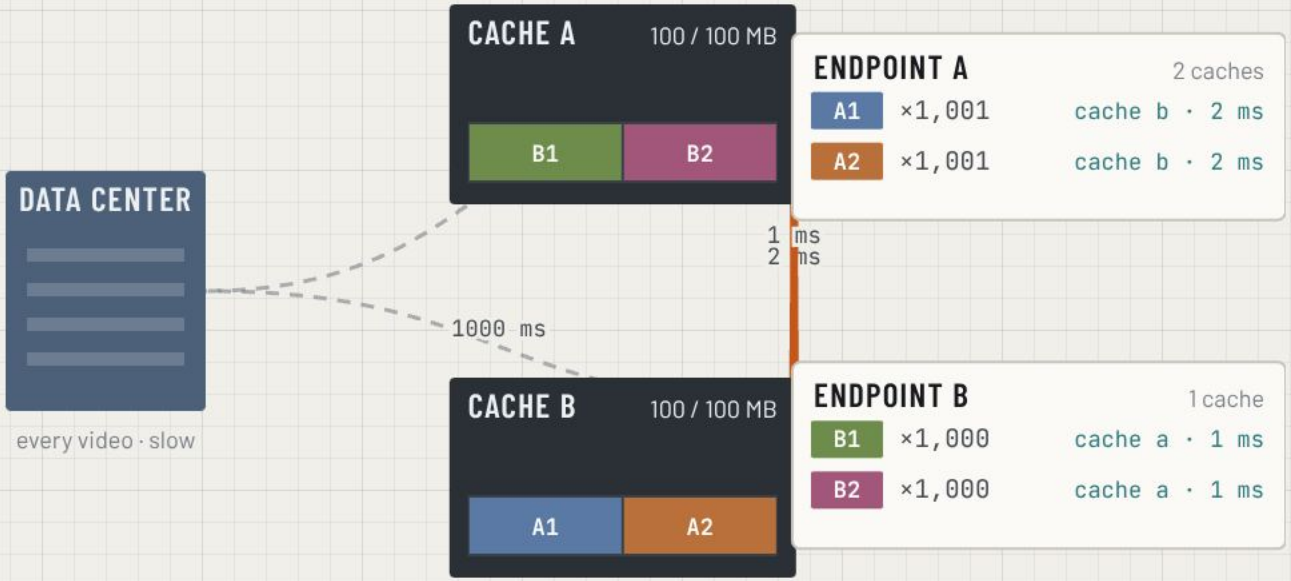
Solved by branch and bound (HiGHS). Small instance: **proven optimum** (zoo: 516,557 in 3 s). Large: a **proven ceiling** (videos: nothing can exceed 620,009).

recorded · exact.py on this instance

```
$ python exact.py trap_gadget.in out.out trap_gadget.in
trap_gadget.out --time-limit 30
6 x vars, 6 y vars
status: Optimization terminated successfully. (HiGHS Status 7:
Optimal)
score 998,499
upper bound 998,499
0s
```

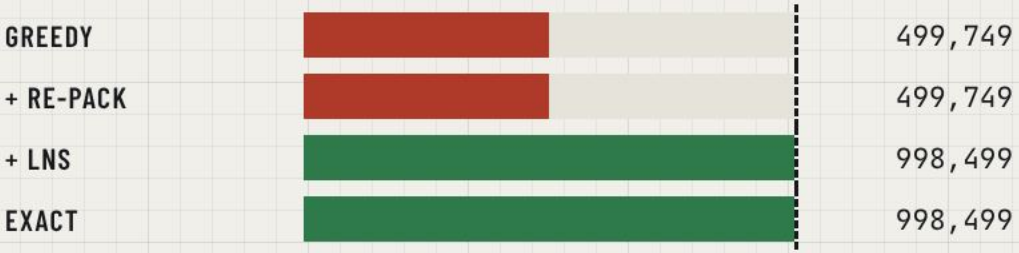
Trap gadget from our generator (models/trap_gadget.in) · solved live · real solver …

06 TWO CACHES AT ONCE, THEN EXACT



Score **998,499** 100.0% of the optimum

LIVE, ON THIS INSTANCE



RUN AGAIN

SAME KIND OF TRAP IN THE CLASS SHEET

DATA SET	GREEDY	NOW
4_2_4_2	67%	100% · proven
6_2_6_2	50%	100% · proven
8_2_8_2	57%	100% · proven
20_2_20_2	53%	100% · proven

Trap gadget from our generator (models/trap_gadget.in) · solved live · real solver …

07 EXPERIMENTAL PROTOCOL

DATA SETS

- **4 official Google data sets** · zoo, videos, trending, kittens
- **29 class data sets** · the shared class sheet, incl. small traps
- **15 local-contest data sets** · scored by the professor
- **12 generated data sets** · tiny → large, sparse → dense, uniform → skewed
- **The statement's example** · our scorer must give 462,500 on its answer first

PIPELINE.md · scores_v1_v2_v3.md · solve.py · local contest sheet

07 EXPERIMENTAL PROTOCOL

DATA SETS

- **4 official Google data sets** · zoo, videos, trending, kittens
- **29 class data sets** · the shared class sheet, incl. small traps
- **15 local-contest data sets** · scored by the professor
- **12 generated data sets** · tiny → large, sparse → dense, uniform → skewed
- **The statement's example** · our scorer must give 462,500 on its answer first

SETTINGS

- **Same machine** for every method: Intel i5-10300H @ 2.5 GHz · 24 GB RAM · Windows 11
- **Greedy:** up to 5 rounds · ≤ 50 swaps per cache · 20 swap candidates
- **Re-pack:** per cache, top 400 videos by saving/MB + top 100 by saving
- **LNS:** 2 caches per step · 5 min per data set
- **Exact solver:** only if ≤ 50,000 variables · 120 s limit

PIPELINE.md · scores_v1_v2_v3.md · solve.py · local contest sheet

07 EXPERIMENTAL PROTOCOL

DATA SETS

- **4 official Google data sets** · zoo, videos, trending, kittens
- **29 class data sets** · the shared class sheet, incl. small traps
- **15 local-contest data sets** · scored by the professor
- **12 generated data sets** · tiny → large, sparse → dense, uniform → skewed
- **The statement's example** · our scorer must give 462,500 on its answer first

SETTINGS

- **Same machine** for every method: Intel i5-10300H @ 2.5 GHz · 24 GB RAM · Windows 11
- **Greedy:** up to 5 rounds · ≤ 50 swaps per cache · 20 swap candidates
- **Re-pack:** per cache, top 400 videos by saving/MB + top 100 by saving
- **LNS:** 2 caches per step · 5 min per data set
- **Exact solver:** only if ≤ 50,000 variables · 120 s limit

LIMITS

- A **time budget** per data set: every step stops on time, a valid answer always exists
- Runtimes come from **one laptop**: a guide, not a benchmark
- LNS is random: **one run** reported per data set
- Annealing compared at **fixed temperatures** only
- "Best in class" = the shared sheet **on 31 October**

PIPELINE.md · scores_v1_v2_v3.md · solve.py · local contest sheet

07 EXPERIMENTAL PROTOCOL

DATA SETS

- **4 official Google data sets** · zoo, videos, trending, kittens
- **29 class data sets** · the shared class sheet, incl. small traps
- **15 local-contest data sets** · scored by the professor
- **12 generated data sets** · tiny → large, sparse → dense, uniform → skewed
- **The statement's example** · our scorer must give 462,500 on its answer first

SETTINGS

- **Same machine** for every method: Intel i5-10300H @ 2.5 GHz · 24 GB RAM · Windows 11
- **Greedy:** up to 5 rounds · ≤ 50 swaps per cache · 20 swap candidates
- **Re-pack:** per cache, top 400 videos by saving/MB + top 100 by saving
- **LNS:** 2 caches per step · 5 min per data set
- **Exact solver:** only if ≤ 50,000 variables · 120 s limit

LIMITS

- A **time budget** per data set: every step stops on time, a valid answer always exists
- Runtimes come from **one laptop**: a guide, not a benchmark
- LNS is random: **one run** reported per data set
- Annealing compared at **fixed temperatures** only
- "Best in class" = the shared sheet **on 31 October**

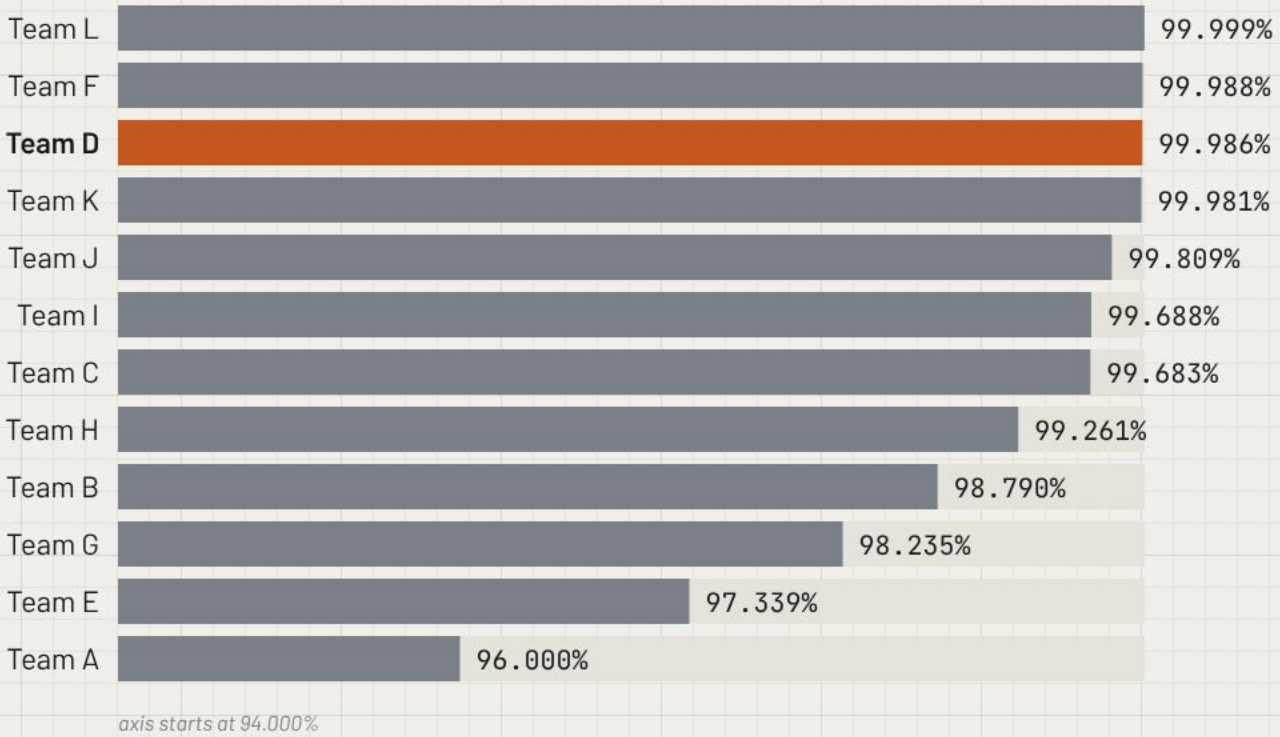
METRICS

- **Official score:** average time saved per request, in µs
- **% of the best:** our score ÷ the best any group reached on that data set
- **Rank**, the professor's three ways: by total, by sum of ranks, by distance to the best
- **Proof:** proven optimum, or the gap to a proven ceiling
- **Runtime** and every output **re-scored** by an independent scorer with a capacity check

PIPELINE.md · scores_v1_v2_v3.md · solve.py · local contest sheet

08 THE OFFICIAL RESULT: LOCAL CONTEST

TOTAL OVER THE 15 DATA SETS · SHARE OF THE BEST POSSIBLE TOTAL (SUM OF BEST SCORES)



axis starts at 94.000%

3rd of 12

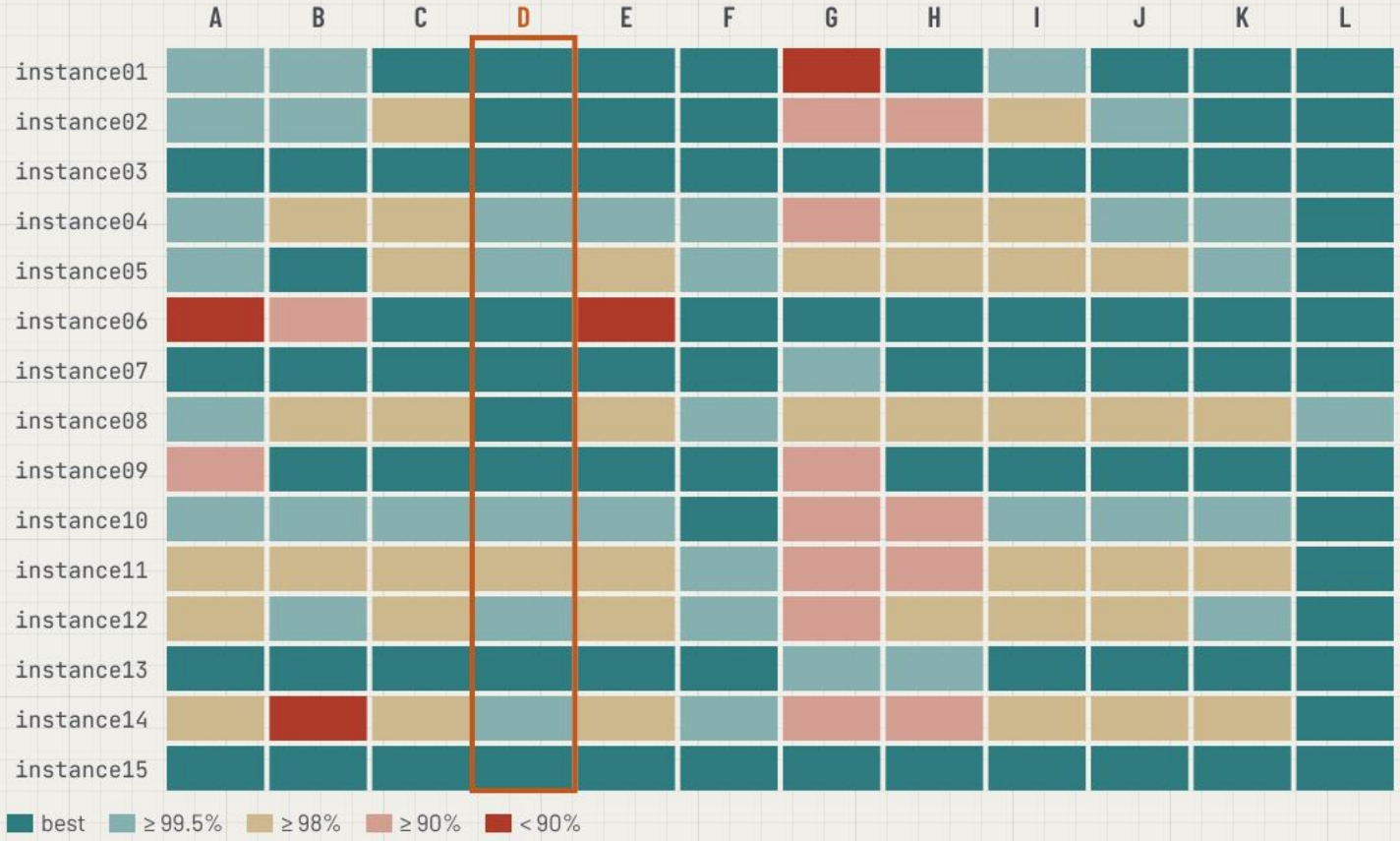
22,875,014 points · **2,870** behind team L (0.0125%)

3rd by all three of the professor's rankings: total (3), sum of ranks (3), distance to the best (3).

Professor's local-contest sheet ('Score', '1re soumission') · 12 teams · 15 data sets

08 THE OFFICIAL RESULT: LOCAL CONTEST

EVERY DATA SET × EVERY TEAM · % OF THE BEST SCORE ON THAT DATA SET (HOVER A CELL)



3rd of 12

22,875,014 points · **2,870** behind team L (0.0125%)

3rd by all three of the professor's rankings: total (3), sum of ranks (3), distance to the best (3).

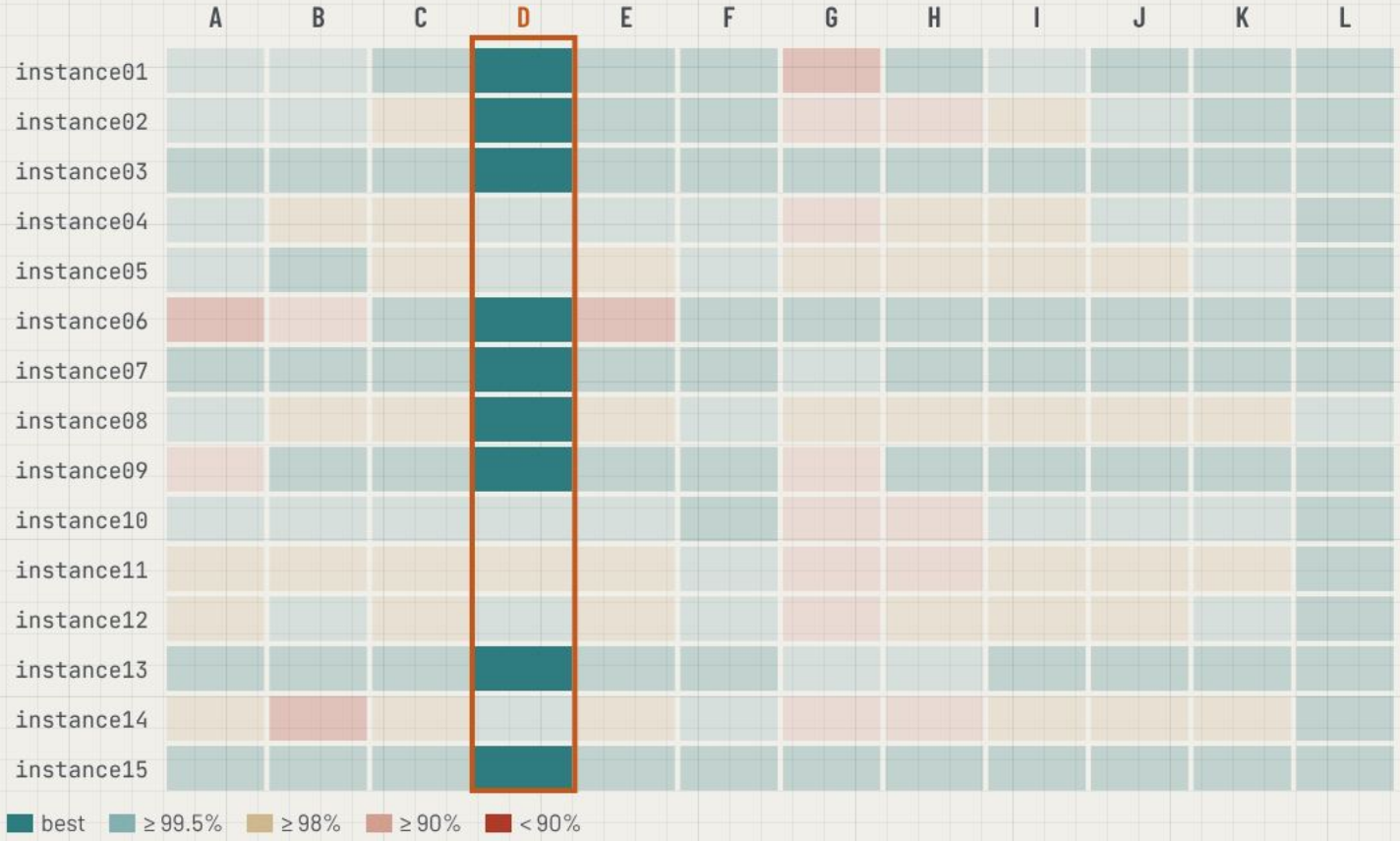
99.3%

our **worst** data set, as a share of the best. 4 of the 11 other teams fall below 90% somewhere.

Professor's local-contest sheet ('Score', '1re soumission') · 12 teams · 15 data sets

08 THE OFFICIAL RESULT: LOCAL CONTEST

EVERY DATA SET × EVERY TEAM · % OF THE BEST SCORE ON THAT DATA SET (HOVER A CELL)



3rd of 12

22,875,014 points · **2,870** behind team L (0.0125%)

3rd by all three of the professor's rankings: total (3), sum of ranks (3), distance to the best (3).

99.3%

our **worst** data set, as a share of the best. 4 of the 11 other teams fall below 90% somewhere.

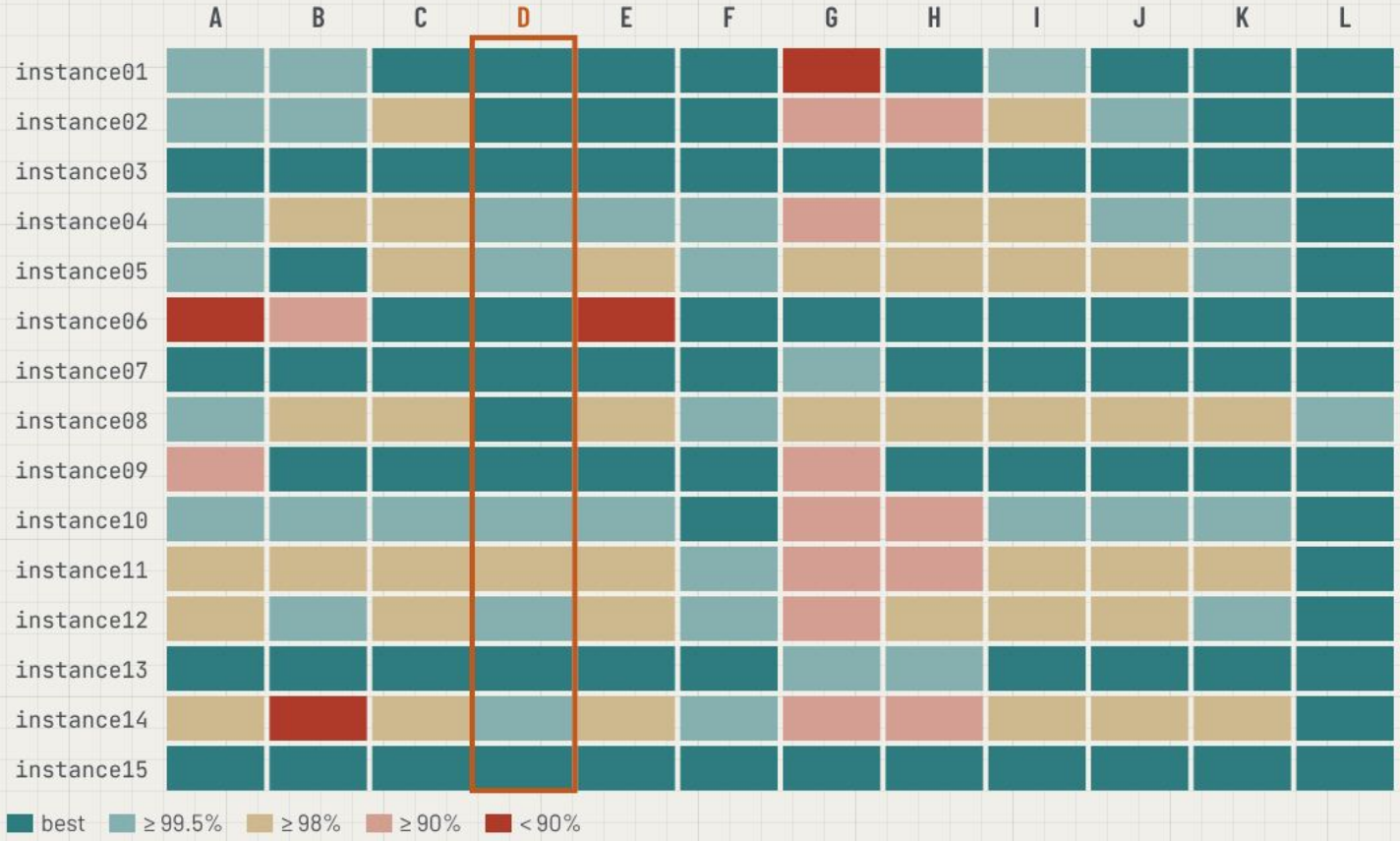
9 of 15

data sets where we have the best score · the **only** team at the best on instance08

Professor's local-contest sheet ('Score', '1re soumission') · 12 teams · 15 data sets

08 THE OFFICIAL RESULT: LOCAL CONTEST

EVERY DATA SET × EVERY TEAM · % OF THE BEST SCORE ON THAT DATA SET (HOVER A CELL)



3rd of 12

22,875,014 points · **2,870** behind team L (0.0125%)

3rd by all three of the professor's rankings: total (3), sum of ranks (3), distance to the best (3).

99.3%

our **worst** data set, as a share of the best. 4 of the 11 other teams fall below 90% somewhere.

9 of 15

data sets where we have the best score · the **only** team at the best on instance08

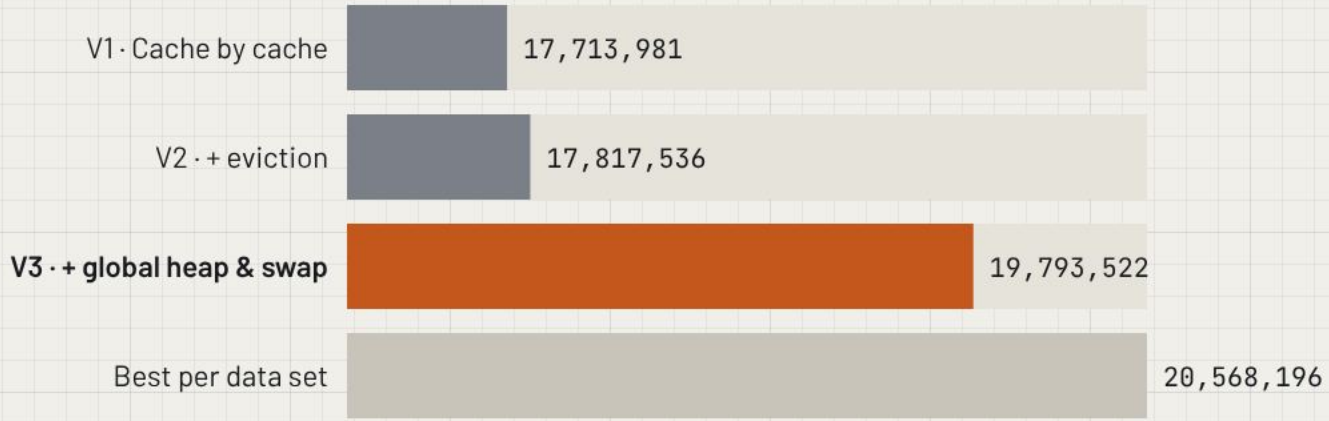
+6,800

from our 1st to our 2nd submission (22,868,214 → 22,875,014). Already 3rd with the first one.

Professor's local-contest sheet ('Score', '1re soumission') · 12 teams · 15 data sets

09 THE CLASS BENCHMARK: FROM 6TH TO 2ND

GREEDY VERSIONS · TOTAL OVER THE 24 CLASS DATA SETS OF THAT TIME



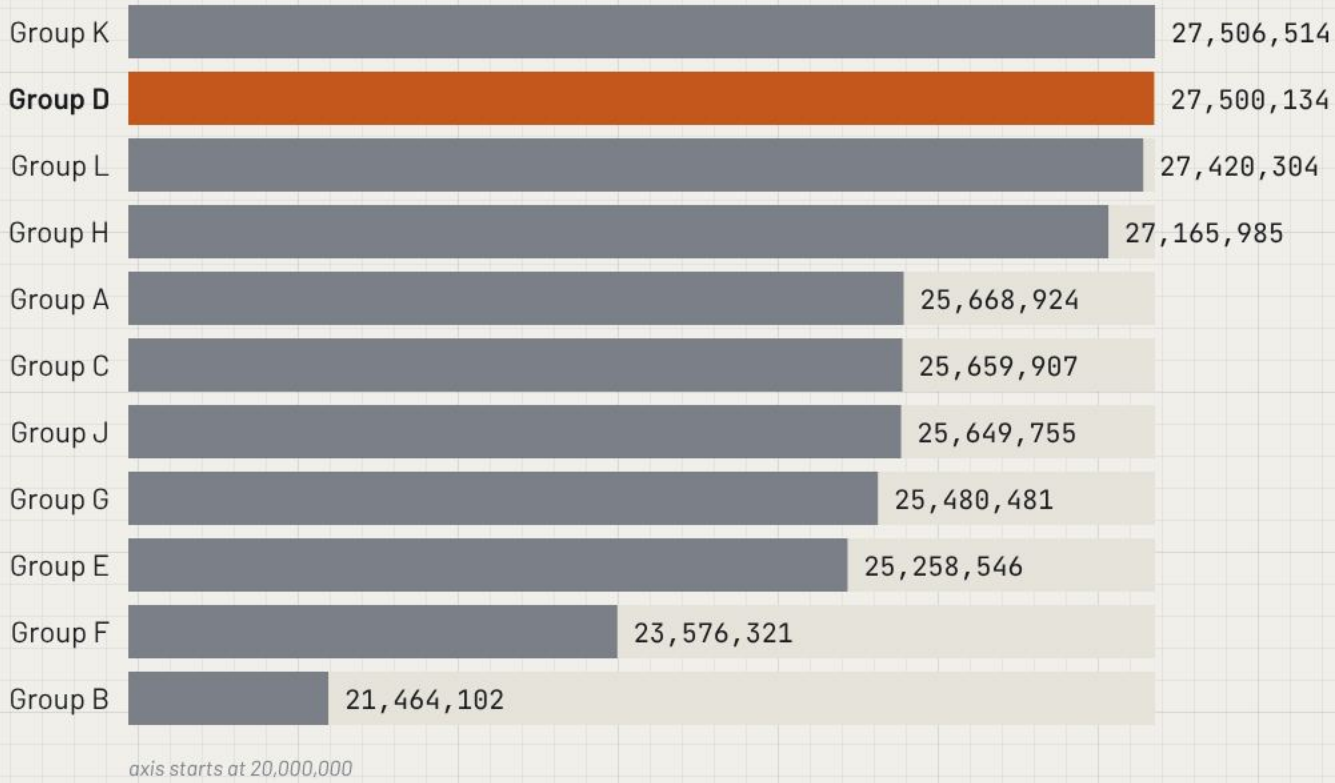
axis starts at 17,000,000

Each version kept the previous ideas and fixed one measured weakness: **+11.7%** from V1 to V3 · 96.2% of the best-per-data-set total.

scores_v1_v2_v3.md · PIPELINE.md §5 · class sheet (31 Oct)

09 THE CLASS BENCHMARK: FROM 6TH TO 2ND

CLASS SHEET TODAY · TOTAL OVER THE 29 CLASS DATA SETS (GROUPS ABOVE 20 M)



+1,865,161

on the 29 class data sets with the full pipeline (25,634,973 → 27,500,134)

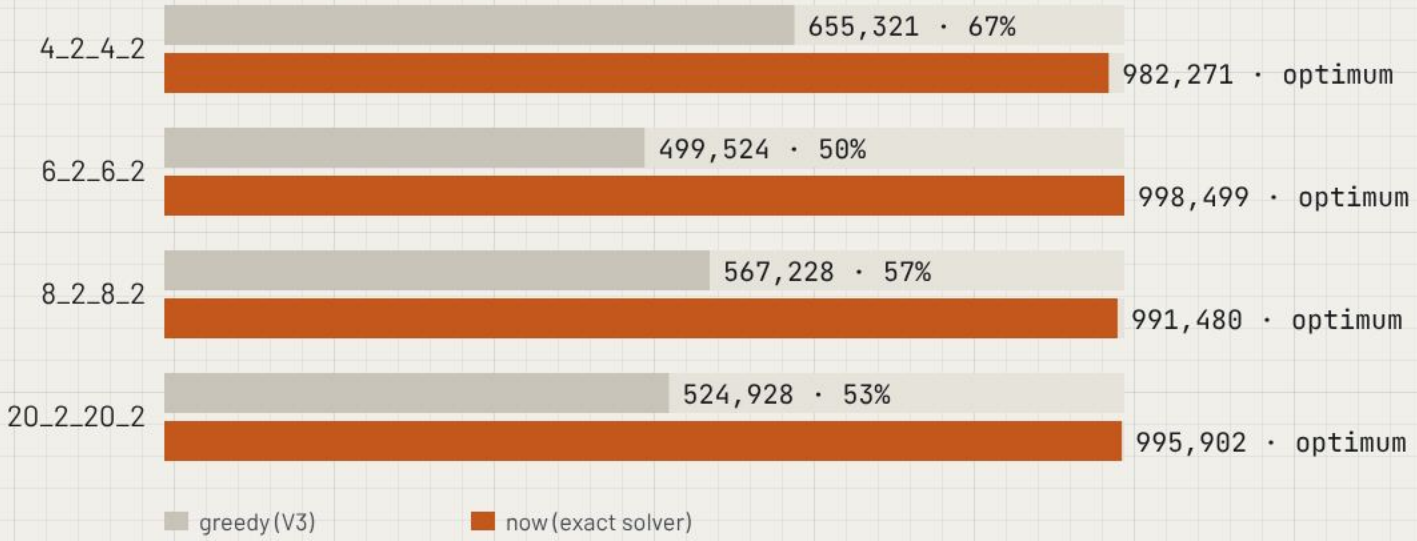
6th → 2nd

rank in the class sheet · 6,380 points behind group K

scores_v1_v2_v3.md · PIPELINE.md §5 · class sheet (31 Oct)

09 THE CLASS BENCHMARK: FROM 6TH TO 2ND

TRAP DATA SETS · GREEDY VS PROVEN OPTIMUM



These four alone: **+1,721,151** of the +1,865,161 gained.

+1,865,161

on the 29 class data sets with the full pipeline (25,634,973 → 27,500,134)

6th → 2nd

rank in the class sheet · 6,380 points behind group K

8

data sets solved to a **proven optimum**: me_at_the_zoo, 2_2_2_1, 4_2_4_2, 6_2_6_2, 8_2_8_2, 20_2_20_2, big, supra

scores_v1_v2_v3.md · PIPELINE.md §5 · class sheet (31 Oct)

09 THE CLASS BENCHMARK: FROM 6TH TO 2ND

GOOGLE DATA SETS · OUR FINAL OUTPUTS, RE-SCORED BY OUR SCORER

DATA SET	GREEDY V3	FINAL	BEST IN CLASS	FINAL STEP
kittens	1,022,115	**1,024,812**	1,025,096	greedy + re-pack + LNS
me_at_the_zoo	507,906	**516,557**	516,557	exact solver · proven optimum
trending_today	499,966	**499,997**	500,000	greedy + re-pack + LNS
videos_worth_spreading	609,385	**613,851**	614,545	greedy + re-pack + LNS
Total	2,639,372	**2,655,217**		→ 2nd of 12 on the Google tab

VIDEOS_WORTH_SPREADING · HOW FAR FROM PERFECT CAN WE BE?



axis starts at 600,000

+1,865,161

on the 29 class data sets with the full pipeline (25,634,973 → 27,500,134)

6th → 2nd

rank in the class sheet · 6,380 points behind group K

8

data sets solved to a **proven optimum**: me_at_the_zoo, 2_2_2_1, 4_2_4_2, 6_2_6_2, 8_2_8_2, 20_2_20_2, big, supra

≤ 6,158

points left on videos_worth_spreading: the exact solver proves no answer can exceed 620,009

scores_v1_v2_v3.md · PIPELINE.md §5 · class sheet (31 Oct)

10 EVERY CHOICE WAS MEASURED

LNS BEAT ANNEALING

9 data sets, 60 s each, same start



Better on **6** data sets vs 2. Annealing only wins with the right temperature, and that temperature changes per data set:

180 S	T = 1	T = 0.001	LNS
videos	**615,635**	610,510	613,548
kittens	1,024,603	**1,024,734**	1,024,706
INSTANCED	1,292,386	1,280,715	**1,297,807**

PIPELINE.md §1, §3, §6 (same machine, same starting point, same time)

10 EVERY CHOICE WAS MEASURED

LNS BEAT ANNEALING

9 data sets, 60 s each, same start

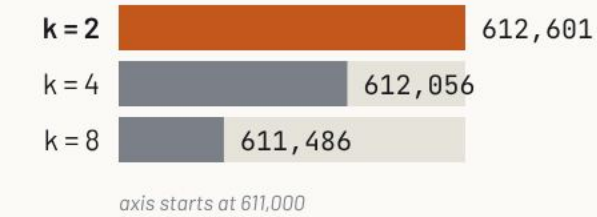


Better on **6** data sets vs 2. Annealing only wins with the right temperature, and that temperature changes per data set:

180 S	T = 1	T = 0.001	LNS
videos	**615,635**	610,510	613,548
kittens	1,024,603	**1,024,734**	1,024,706
INSTANCED	1,292,386	1,280,715	**1,297,807**

TWO CACHES, FIVE MINUTES

LNS group size · videos, 20 s



axis starts at 611,000

More time on videos: gain over re-pack



Small groups rebuild faster, so more steps fit. Gains flatten: 5 minutes is the sweet spot.

PIPELINE.md §1, §3, §6 (same machine, same starting point, same time)

10 EVERY CHOICE WAS MEASURED

LNS BEAT ANNEALING

9 data sets, 60 s each, same start

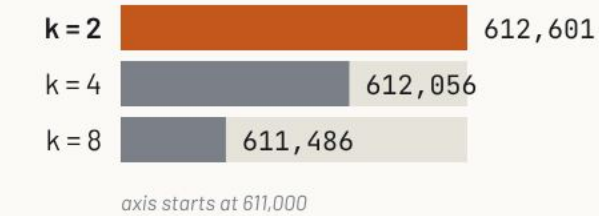


Better on **6** data sets vs 2. Annealing only wins with the right temperature, and that temperature changes per data set:

180 S	T = 1	T = 0.001	LNS
videos	**615,635**	610,510	613,548
kittens	1,024,603	**1,024,734**	1,024,706
INSTANCED	1,292,386	1,280,715	**1,297,807**

TWO CACHES, FIVE MINUTES

LNS group size · videos, 20 s



More time on videos: gain over re-pack



Small groups rebuild faster, so more steps fit. Gains flatten: 5 minutes is the sweet spot.

WHEN TO RUN THE EXACT SOLVER

Size of the integer program (assignment variables)

DATA SET	VARIABLES	EXACT SOLVER
zoo	261	wins
zipF	2,539	wins
regional	2,943	wins
big	8,762	wins
videos	210,540	loses
INSTANCED	929,425	loses

Threshold set in between: **50,000** variables.

Also measured: re-pack starting from the greedy beats starting from empty caches (videos 610,505 vs 606,631 · zoo 512,115 vs 500,513).

PIPELINE.md §1, §3, §6 (same machine, same starting point, same time)

11 SWOT

STRENGTHS

- **No step can lower the score**: lazy heap, safe eviction, keep-if-not-worse
- **8 proven optima**, and a proven ceiling where we can't solve exactly
- **Consistent**: never below 99.3% of the best in the local contest
- **Fast start**: kittens greedy in 13 s, always a valid answer

Our measurements (scenes 08–10) · class sheet · local contest

11 SWOT

STRENGTHS

- **No step can lower the score**: lazy heap, safe eviction, keep-if-not-worse
- **8 proven optima**, and a proven ceiling where we can't solve exactly
- **Consistent**: never below 99.3% of the best in the local contest
- **Fast start**: kittens greedy in 13 s, always a valid answer

WEAKNESSES

- The greedy alone **falls into traps** (50–67% of the optimum)
- Re-pack and LNS are **local**: one or two caches at a time
- LNS gains **flatten** on large data sets after a few minutes
- The exact solver stops at **50,000 variables**

Our measurements (scenes 08–10) · class sheet · local contest

11 SWOT

STRENGTHS

- **No step can lower the score**: lazy heap, safe eviction, keep-if-not-worse
- **8 proven optima**, and a proven ceiling where we can't solve exactly
- **Consistent**: never below 99.3% of the best in the local contest
- **Fast start**: kittens greedy in 13 s, always a valid answer

WEAKNESSES

- The greedy alone **falls into traps** (50–67% of the optimum)
- Re-pack and LNS are **local**: one or two caches at a time
- LNS gains **flatten** on large data sets after a few minutes
- The exact solver stops at **50,000 variables**

OPPORTUNITIES

- Up to **6,158** points left on videos (proven ceiling)
- Run LNS on **several cores** in parallel
- **Split** large integer programs so the exact solver reaches big data sets
- Reuse the class console as a **teaching tool**

Our measurements (scenes 08–10) · class sheet · local contest

11 SWOT

STRENGTHS

- **No step can lower the score**: lazy heap, safe eviction, keep-if-not-worse
- **8 proven optima**, and a proven ceiling where we can't solve exactly
- **Consistent**: never below 99.3% of the best in the local contest
- **Fast start**: kittens greedy in 13 s, always a valid answer

WEAKNESSES

- The greedy alone **falls into traps** (50–67% of the optimum)
- Re-pack and LNS are **local**: one or two caches at a time
- LNS gains **flatten** on large data sets after a few minutes
- The exact solver stops at **50,000 variables**

OPPORTUNITIES

- Up to **6,158** points left on videos (proven ceiling)
- Run LNS on **several cores** in parallel
- **Split** large integer programs so the exact solver reaches big data sets
- Reuse the class console as a **teaching tool**

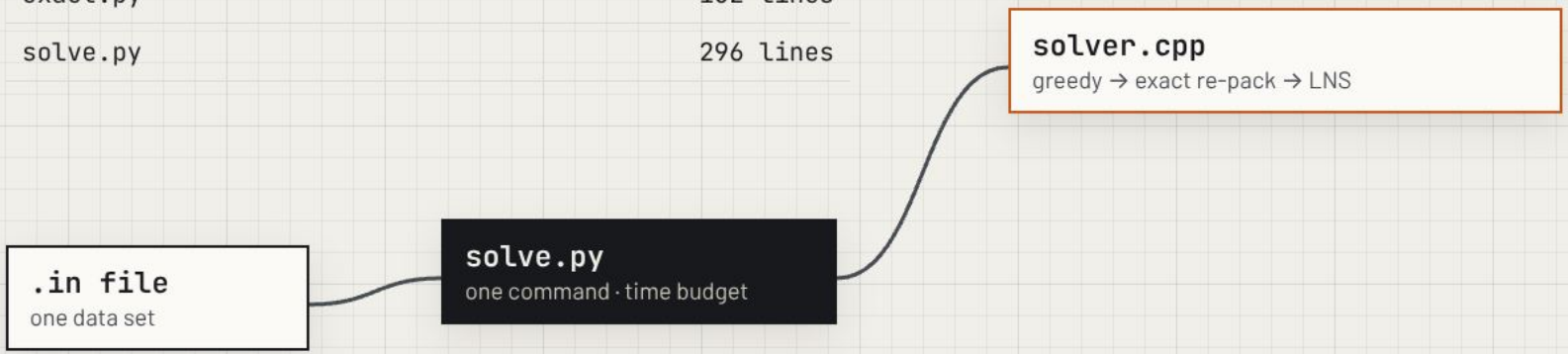
THREATS

- The top 3 of the contest are within **0.013%**: tiny gaps decide the rank
- Other teams ahead on some data sets (local contest: 6 of 15)
- The exact solver is **exponential** in the worst case
- Tight time limits on million-request inputs leave LNS little time

Our measurements (scenes 08–10) · class sheet · local contest

12 HOW THE FINAL SUBMISSION IS BUILT

solution.py	745 lines
solver.cpp	819 lines
exact.py	102 lines
solve.py	296 lines



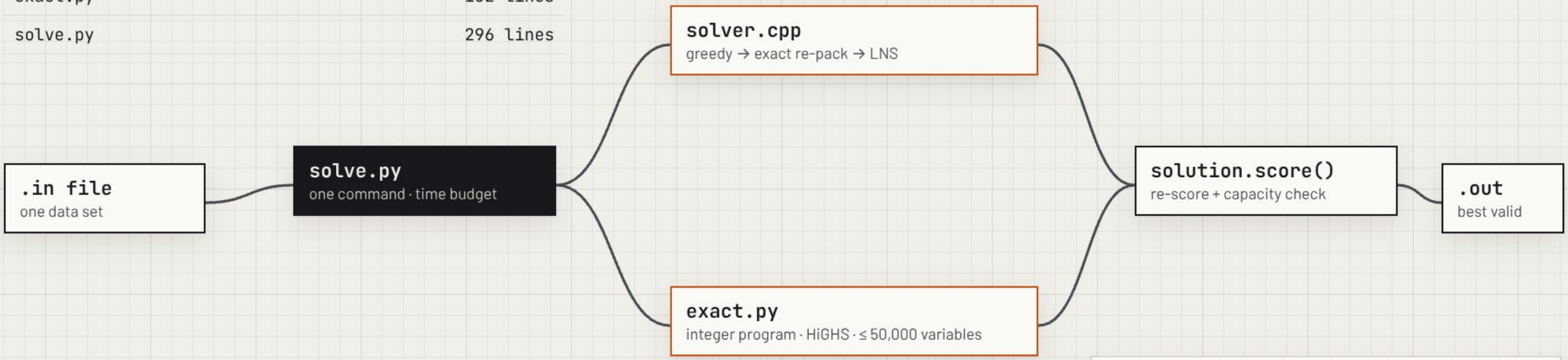
CLICK A BOX

Each box is one file of the repository.

Repository: solve.py, solver.cpp, exact.py, solution.py, runner/, api/, ui/

12 HOW THE FINAL SUBMISSION IS BUILT

solution.py	745 lines
solver.cpp	819 lines
exact.py	102 lines
solve.py	296 lines



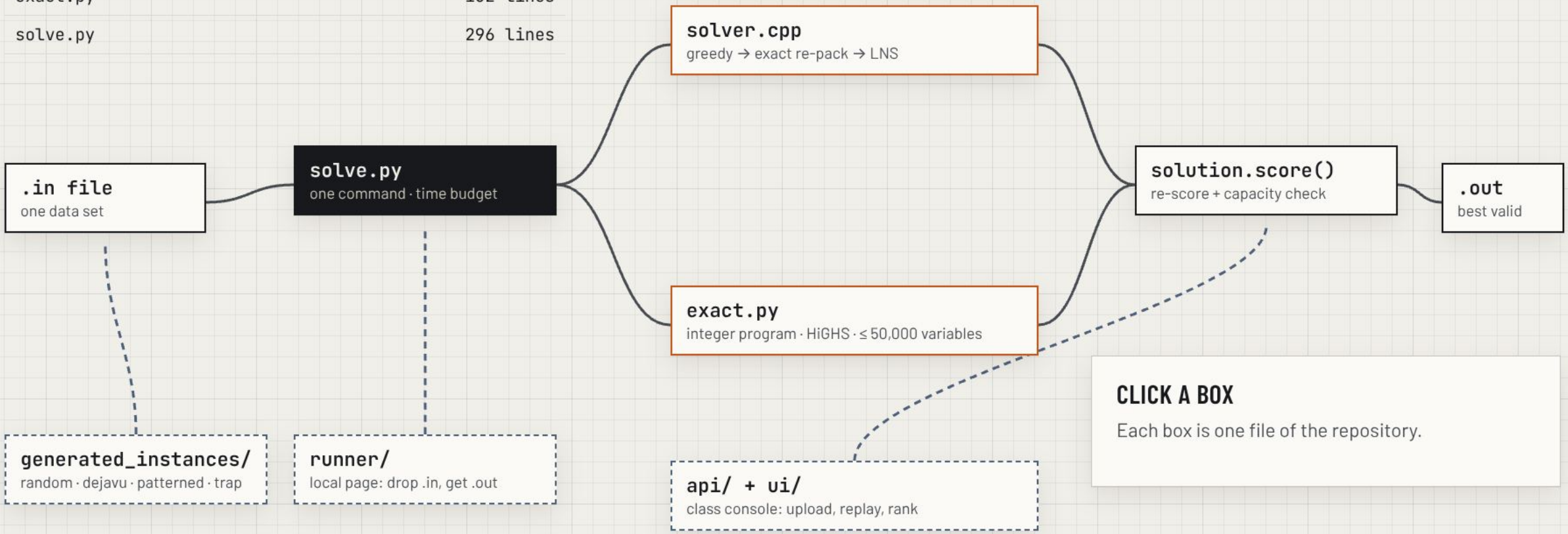
CLICK A BOX

Each box is one file of the repository.

Repository: solve.py, solver.cpp, exact.py, solution.py, runner/, api/, ui/

12 HOW THE FINAL SUBMISSION IS BUILT

solution.py	745 lines
solver.cpp	819 lines
exact.py	102 lines
solve.py	296 lines



CLICK A BOX

Each box is one file of the repository.

Repository: solve.py, solver.cpp, exact.py, solution.py, runner/, api/, ui/

13 IMPLEMENTATION CHOICES

PYTHON FIRST, THEN C++

Same moves in both: identical scores on the 4 Google data sets and 30 of 31 class ones (the last differs by 1 point, a tie). Kittens: 362 s → 13 s.

ONLY LOOK AT REAL DEMAND

A cache only scores the videos its own endpoints request. The work grows with links × requests, never with videos × caches.

PIPELINE.md §1–§2, §6 · solver.cpp · git log (19 Oct)

13 IMPLEMENTATION CHOICES

PYTHON FIRST, THEN C++

Same moves in both: identical scores on the 4 Google data sets and 30 of 31 class ones (the last differs by 1 point, a tie). Kittens: 362 s → 13 s.

ONLY LOOK AT REAL DEMAND

A cache only scores the videos its own endpoints request. The work grows with links × requests, never with videos × caches.

LAZY MAX-HEAP

Gains only ever go down, so a stored value is an upper bound. Only the pair on top is re-scored: placed if unchanged, pushed back otherwise.

BEST AND SECOND-BEST LATENCY

Stored per request, with which cache gives the best. "Fastest from another cache" is then a constant-time lookup during re-pack.

PIPELINE.md §1–§2, §6 · solver.cpp · git log (19 Oct)

13 IMPLEMENTATION CHOICES

PYTHON FIRST, THEN C++

Same moves in both: identical scores on the 4 Google data sets and 30 of 31 class ones (the last differs by 1 point, a tie). Kittens: 362 s → 13 s.

ONLY LOOK AT REAL DEMAND

A cache only scores the videos its own endpoints request. The work grows with links × requests, never with videos × caches.

LAZY MAX-HEAP

Gains only ever go down, so a stored value is an upper bound. Only the pair on top is re-scored: placed if unchanged, pushed back otherwise.

BEST AND SECOND-BEST LATENCY

Stored per request, with which cache gives the best. "Fastest from another cache" is then a constant-time lookup during re-pack.

BITSET KNAPSACK, SIZES ÷ GCD

About 500 candidates per cache; one bit per (video, MB) cell. Dividing sizes and capacity by their gcd gives the same answer, up to 5,000× faster on equal-size data sets.

ALWAYS A VALID ANSWER

The first greedy round always completes; every later step stops on time. A 120 s limit stopped at 120.2 s on a data set near the official limits.

PIPELINE.md §1–§2, §6 · solver.cpp · git log (19 Oct)

13 IMPLEMENTATION CHOICES

PYTHON FIRST, THEN C++

Same moves in both: identical scores on the 4 Google data sets and 30 of 31 class ones (the last differs by 1 point, a tie). Kittens: 362 s → 13 s.

ONLY LOOK AT REAL DEMAND

A cache only scores the videos its own endpoints request. The work grows with links × requests, never with videos × caches.

LAZY MAX-HEAP

Gains only ever go down, so a stored value is an upper bound. Only the pair on top is re-scored: placed if unchanged, pushed back otherwise.

BEST AND SECOND-BEST LATENCY

Stored per request, with which cache gives the best. "Fastest from another cache" is then a constant-time lookup during re-pack.

BITSET KNAPSACK, SIZES ÷ GCD

About 500 candidates per cache; one bit per (video, MB) cell. Dividing sizes and capacity by their gcd gives the same answer, up to 5,000× faster on equal-size data sets.

ALWAYS A VALID ANSWER

The first greedy round always completes; every later step stops on time. A 120 s limit stopped at 120.2 s on a data set near the official limits.

TRUST NOTHING

Every .out is re-scored by an independent scorer and every cache capacity is checked before it can be submitted.

LAST ROUND OF TUNING

Added with our 2nd local-contest submission: extra moves that swap two videos between neighbouring caches or replace one, with optional annealing. Off by default.

PIPELINE.md §1–§2, §6 · solver.cpp · git log (19 Oct)

14 SINCE OUR LAST PRESENTATION

LAST TIME

4th of 9

2,639,372 on the Google data sets · Python greedy only →

WHAT WE SAID WE WOULD DO NEXT

"Exact packing for small caches"
our biggest gap (zoo)

"Simulated annealing"
escape local optima

"Faster code"
kittens took ~6 min in Python

"Pick the best strategy"
per type of instance

Previous presentation (slides 8–10) · class sheet · local contest

14 SINCE OUR LAST PRESENTATION

LAST TIME

4th of 9

2,639,372 on the Google data sets · Python greedy only →

WHAT WE SAID WE WOULD DO NEXT

"Exact packing for small caches" our biggest gap (zoo)	✓ **Exact per-cache re-pack** `zoo now proven optimal`
"Simulated annealing" escape local optima	
"Faster code" kittens took ~6 min in Python	
"Pick the best strategy" per type of instance	

Previous presentation (slides 8–10) · class sheet · local contest

14 SINCE OUR LAST PRESENTATION

LAST TIME

4th of 9

2,639,372 on the Google data sets · Python greedy only →

WHAT WE SAID WE WOULD DO NEXT

"Exact packing for small caches" our biggest gap (zoo)	✓ **Exact per-cache re-pack** `zoo now proven optimal`
"Simulated annealing" escape local optima	✓ **Annealing and LNS, tested side by side** `LNS won 6 of 9 instances`
"Faster code" kittens took ~6 min in Python	
"Pick the best strategy" per type of instance	

Previous presentation (slides 8–10) · class sheet · local contest

14 SINCE OUR LAST PRESENTATION

LAST TIME

4th of 9

2,639,372 on the Google data sets · Python greedy only →

WHAT WE SAID WE WOULD DO NEXT

"Exact packing for small caches" our biggest gap (zoo)	**Exact per-cache re-pack** `zoo now proven optimal`
"Simulated annealing" escape local optima	**Annealing and LNS, tested side by side** `LNS won 6 of 9 instances`
"Faster code" kittens took ~6 min in Python	**C++ port, same moves** `kittens greedy in 13 s`
"Pick the best strategy" per type of instance	

Previous presentation (slides 8–10) · class sheet · local contest

14 SINCE OUR LAST PRESENTATION

LAST TIME

4th of 9

2,639,372 on the Google data sets · Python greedy only

→

NOW

2nd of 12

on the class benchmark · 3rd of 12 in the local contest

WHAT WE SAID WE WOULD DO NEXT

"Exact packing for small caches" our biggest gap (zoo)	**Exact per-cache re-pack** `zoo now proven optimal`
"Simulated annealing" escape local optima	**Annealing and LNS, tested side by side** `LNS won 6 of 9 instances`
"Faster code" kittens took ~6 min in Python	**C++ port, same moves** `kittens greedy in 13 s`
"Pick the best strategy" per type of instance	**One command runs every method** `keeps the best valid answer`

Previous presentation (slides 8–10) · class sheet · local contest

15 CONCLUSION

LAYER	EXACT?	CHANGES PER STEP	FIXES
Greedy (heap, evict, swap)	no	one (cache, video) pair	a good start, fast
Exact re-pack	yes, one cache	a whole cache	the ratio rule misses combinations
Large neighbourhood search	yes, two caches	two linked caches	re-pack is stuck one cache at a time
Exact solver	yes, globally	the whole solution	traps · gives a proven ceiling

Each layer fixes one **measured** weakness of the one below, and none can make the answer worse.

Scenes 03–14

15 CONCLUSION

LAYER	EXACT?	CHANGES PER STEP	FIXES
Greedy (heap, evict, swap)	no	one (cache, video) pair	a good start, fast
Exact re-pack	yes, one cache	a whole cache	the ratio rule misses combinations
Large neighbourhood search	yes, two caches	two linked caches	re-pack is stuck one cache at a time
Exact solver	yes, globally	the whole solution	traps · gives a proven ceiling

Each layer fixes one **measured** weakness of the one below, and none can make the answer worse.

BUILD FAST, IMPROVE WITH EXACT SUB-PROBLEMS, SOLVE EXACTLY WHAT IS SMALL ENOUGH.

3rd of 12
local contest (official)

2nd of 12
class benchmark, 29 data sets

8
proven optima

Scenes 03–14

QUESTIONS?

Whiteboard welcome. Backup scenes: a live playground, why each step is safe, and the cost of each step.

Group D Kacem · Daniel · Marouane · Dhia

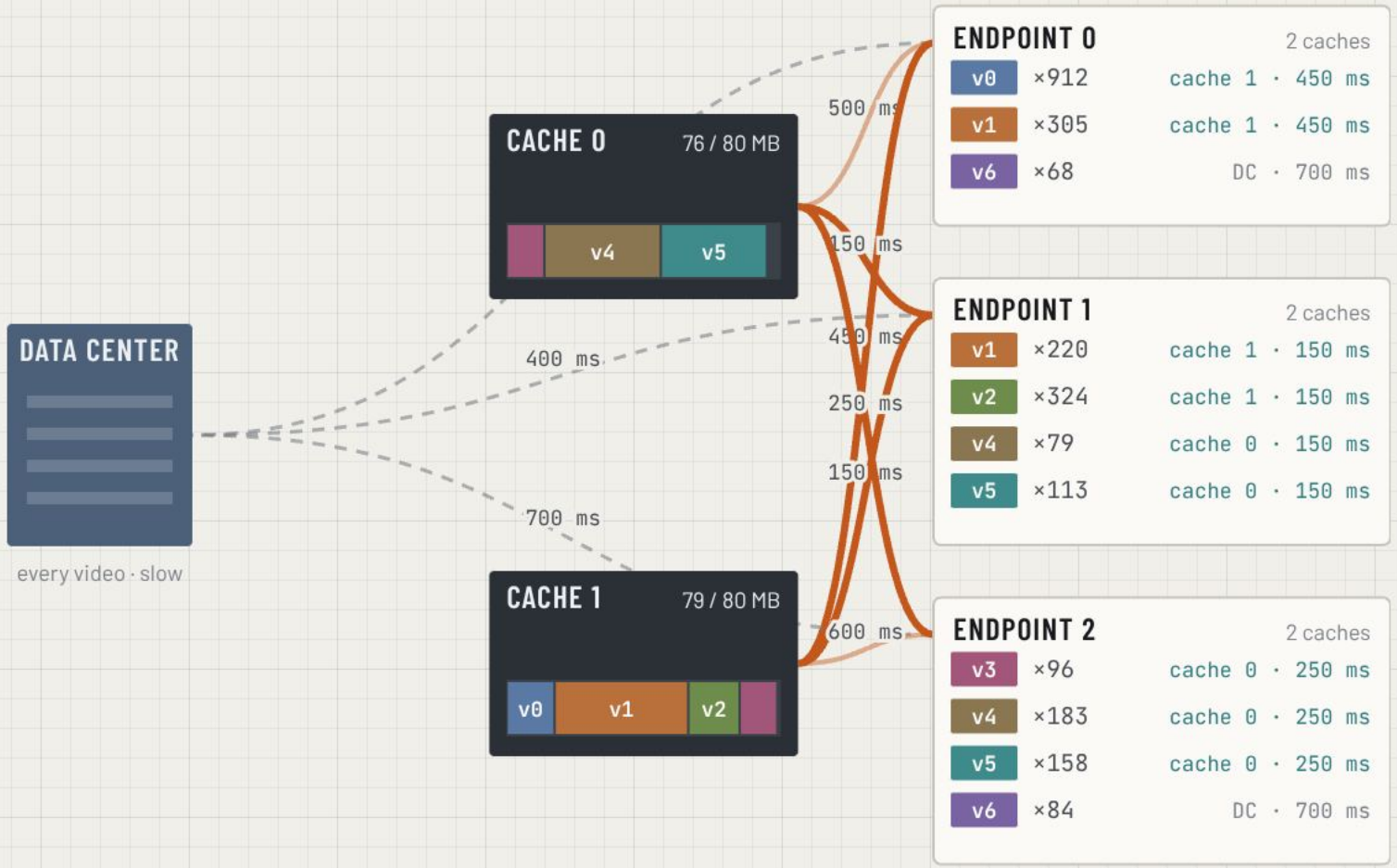
github.com/HashCode-2017/hashcode_2017

Scenes 03–14

16 BACKUP · PLAYGROUND

VIDEOS 7 | ENDPOINTS 3 | CACHES 2 | CACHE MB 80 | DEMAND SKEW 1.2

NEW INSTANCE | COPY .IN | seed 176



EACH LAYER, ON THIS INSTANCE

GREEDY	230,350
+ RE-PACK	256,215
+ LNS	269,433
OPTIMUM	269,433

Showing **lns**: 269,433 · 100.0% of the optimum

GREEDY | RE-PACK | LNS | OPTIMUM

Random small instances (ours) · same solver as scenes 04–06

17 BACKUP · WHY EACH STEP IS SAFE

WHY THE LAZY HEAP IS RIGHT

Adding a copy can only lower the other pairs' gains, never raise them. So a value stored in the heap is an upper bound. When the top pair is re-scored and is unchanged, nothing below can beat it: placing it is exactly the "best now" choice.

WHY RE-PACK IS AN EXACT KNAPSACK

Freeze every other cache. For a request, the best latency from elsewhere is then fixed, so the saving of video v in cache c is a fixed number g(v). Different videos in one cache do not interact. Maximise Σ g(v) under Σ size ≤ X: a 0/1 knapsack, solved exactly by DP.

WHY EVICTION NEVER LOWERS THE SCORE

A copy is removed only if every endpoint of that cache asking for the video already gets it at least as fast from another cache (or the data center). No request changes latency, so the score is identical right after the sweep.

WHY LNS NEVER LOWERS THE SCORE

It remembers the old contents and the saving S of every endpoint touching the emptied caches. Only those endpoints can change. It keeps the rebuild only if the new saving ≥ S, otherwise it puts the old contents back.

WHERE THE CEILING COMES FROM

Branch and bound solves relaxations of the integer program. Any relaxed optimum is ≥ the true optimum, so the best bound found is a ceiling no submission can pass: 620,009 on videos_worth_spreading.

WHY THE PROBLEM IS NP-HARD

With one cache and one endpoint, it is exactly a 0/1 knapsack (sizes = weights, savings = values). With many caches sharing endpoints it also contains facility location.

PIPELINE.md §1–§4 · SOLUTION.md §3

18 BACKUP · COST OF EACH STEP

STEP	TIME	WHY
Read and index	$O(V + E + L + R)$	one pass builds: requests per endpoint, endpoints per cache, requesters per video
Greedy fill (per round)	$O(W + (H + U)(\log H + r_{max}))$	score each (cache, video) pair once; each pop re-scores one pair
Swap (per round)	$O(\Sigma_c\ (1 + S_c)\ W_c\ K_{max})$	per cache, once plus once after each swap (≤ 50)
Re-pack one cache	$O(W_c + m \log m + m \cdot X)$	gains, pick m candidates, DP (bitset: m·X/64 words)
LNS step (k = 2)	$O(k \cdot \text{sweeps} \cdot (W_c + m \cdot X))$	only endpoints touching the k caches are re-scored
Exact solver	$\text{exponential (worst case)}$	branch and bound; seconds when ≤ 10,000 variables

SYMBOLS

V, E, R, C, X: videos, endpoints, request lines, caches, cache MB · L: endpoint–cache links · K_{max}: most links of one endpoint · $W = \Sigma_e K_e \cdot Q_e$: (cache, endpoint, requested video) triples · H: pairs in the heap · U: pushes back · r_v: endpoints asking for v · m: knapsack candidates (≈ 500)

DATA SET	V	E	R	C
me_at_the_zoo	100	10	100	10
videos_worth_spreading	10,000	100	100,000	100
trending_today	10,000	100	100,000	100
kittens	10,000	1,000	200,000	500

README.md · Complexity · PIPELINE.md §1–§4